



福建理工大学
FUJIAN UNIVERSITY OF TECHNOLOGY

毕业设计（论文）

题目：基于大语言模型的缺陷检测 AGENT 设计与实现

学生：颜益炜

指导老师：郑积仕 副教授

学院：人工智能与交通工程学院

专业：信息管理与信息系统

班级：信管（闽台）2202

学号：3221806228

福建理工大学教务处制



福建理工大学本科毕业设计（论文）作者承诺以及保证书

作者在此郑重承诺：本篇毕业设计（论文）中所记载的内容均真实且可靠；如果出现弄虚作假、抄袭等情形，作者愿意承担由此产生的全部责任。

学生签名：

年 月 日

福建理工大学本科毕业设计（论文）指导教师承诺以及保证书

本人在此郑重承诺：已依照有关规定，对本篇毕业设计（论文）的选题以及内容开展了指导与审核工作；所提交的毕业设计（论文）终稿，与上传至“大学生论文管理系统”进行检测的电子文档保持一致，未发现弄虚作假、抄袭等现象，本人愿承担指导教师应当承担的相关责任。

指导教师签名：

年 月 日

基于大语言模型的缺陷检测 AGENT 设计与实现

摘 要

随着人工智能的发展，高精度光学元件在算力中心的需求越来越大，并行差动阵列共聚焦三维测量（Dispersion Array Confocal 3D, DAC-3D）是一套用于检测高精度光学元件的检测装备。该检测装备在实际应用过程当中，需要依次完成参数设置、扫描区域确认、运行状态查看、检测结果判读以及异常处理等工作。传统界面虽然能够支撑基础检测控制，但在知识检索、自然语言操作、结果解释、操作确认以及过程追踪等方面仍存在不足。针对上述问题，本文设计并实现了一套面向 DAC-3D 应用场景的大语言模型智能交互助手。该系统不改变主检测算法和主检测流程，而是在主系统之外构建受知识约束、受工具网关控制并且能够追踪验证的辅助交互层。系统采用前后端分离和模块化设计，后端以统一 Agent 运行时作为请求入口，结合知识库检索、上下文工程、技能选择、会话记忆、工具网关、安全审查、DAC-3D 适配、结果解析以及 Trace/Eval 等模块处理用户请求；前端基于 React 与 Vite 构建交互界面，用于展示回答内容、来源依据、结构化命令、运行状态以及命令确认结果。

在功能实现方面，系统针对参数问答和操作指导，依据本地知识库及相关技能说明组织回答；针对操作类请求，先生成命令预览，再依次经过策略校验、路径白名单、风险分类以及显式确认；针对结果解释，则以 DAC-3D 主系统已有检测输出和缺陷规则为基础生成说明。测试与验证工作围绕功能、接口、异常边界、安全策略、语法检查、场景化运行以及用户体验等方面展开，覆盖知识库构建、检索问答、命令预览、状态查询、结果解释以及 Web 交互等链路；在完成 Agent 化改造后，又补充了本地 Agent 回归用例和安全红队用例，用于验证状态查询、命令预览、批准提交、记忆写入候选以及 Prompt Injection 拦截等场景。验证结果表明，该方案可以在 DAC-3D 场景下形成能够运行的智能交互助手原型。

关键词：大语言模型，Agent 运行时，检索增强生成，工业检测软件，DAC-3D

Design and Implementation of a Defect Detection Agent Based on a Large Language Model

Abstract

With the development of artificial intelligence, the demand for high-precision optical components in computing centers has been increasing. Dispersion Array Confocal 3D (DAC-3D) is an inspection system used for detecting high-precision optical components. In practical applications, this equipment requires users to complete parameter setting, scan-area confirmation, runtime status checking, inspection-result interpretation, and exception handling in sequence. Although traditional interfaces can support basic inspection control, they still have limitations in knowledge retrieval, natural-language operation, result explanation, operation confirmation, and process tracing. To address these problems, this thesis designs and implements a large-language-model-based intelligent interaction assistant for DAC-3D application scenarios. The system does not change the main inspection algorithm or the main inspection workflow. Instead, it builds an auxiliary interaction layer outside the main system that is constrained by knowledge, controlled by a tool gateway, and traceable for verification. The system adopts a front-end and back-end separated modular design. The back end uses a unified Agent runtime as the request entry and combines knowledge-base retrieval, context engineering, skill selection, conversation memory, tool gateway, safety review, DAC-3D adaptation, result parsing, and Trace/Eval modules to process user requests. The front end is built with React and Vite to present answers, source evidence, structured commands, runtime status, and command confirmation results.

In terms of functional implementation, the system organizes answers for parameter questions and operation guidance based on the local knowledge base and related skill descriptions. For operation requests, it first generates command previews and then performs policy validation, path allowlist checking, risk classification, and explicit confirmation in sequence. For result interpretation, it generates explanations based on the existing inspection outputs of the DAC-3D main system and defect rules. Testing and validation were conducted from the perspectives of functionality, interfaces, abnormal boundaries, safety policies, syntax checking, scenario-based operation, and user experience, covering workflows such as knowledge-base construction, retrieval-based question answering, command preview, status query, result interpretation, and Web interaction. After the Agent-based transformation, local Agent regression cases and security red-team cases were further added to verify scenarios including status query, command preview, approval submission, memory write candidates, and Prompt Injection blocking. The validation results show that the proposed approach can form a runnable intelligent interaction assistant prototype for the DAC-3D scenario.

Key words: Large language model, Agent runtime, Retrieval-augmented generation, Industrial inspection software, DAC-3D

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景	1
1.1.1 工业检测软件交互背景	1
1.1.2 DAC-3D 缺陷检测系统使用痛点	1
1.2 国内外研究现状	2
1.2.1 大语言模型与检索增强研究现状	2
1.2.2 任务型对话与工业数字助手研究	3
1.2.3 研究评述与本文定位	4
1.3 研究目的与意义	4
1.4 研究内容与论文结构	5
第 2 章 相关技术与理论基础	6
2.1 大语言模型与任务型交互	6
2.2 检索增强生成与混合检索	7
2.3 意图识别与结构化命令生成	8
2.4 DAC-3D 主系统桥接与结果解析技术	9
2.5 FastAPI、React 与前后端分离架构	11
2.6 本章小结	12
第 3 章 系统需求分析	13
3.1 DAC-3D 检测业务流程分析	13
3.1.1 检测准备阶段	13
3.1.2 检测执行阶段	13
3.1.3 结果分析阶段	14
3.2 用户角色与使用场景分析	14
3.2.1 操作人员角色	14
3.2.2 系统维护人员角色	15
3.2.3 DAC-3D 主系统与助手服务边界	15
3.3 功能需求分析	15
3.4 非功能需求分析	17
3.5 用例分析	18
3.5.1 需求边界与验收依据	18
3.5.2 用例覆盖与测试追踪	20

3.6 本章小结	20
第 4 章 系统总体设计	21
4.1 系统设计目标与设计原则	22
4.2 系统总体架构设计	23
4.3 系统功能流程设计	24
4.4 核心数据结构设计	25
4.5 接口设计	27
4.6 部署与集成设计	28
4.6.1 本地独立运行模式	28
4.6.2 DAC-3D 主系统跳转网页助手模式	28
4.6.3 状态文件桥接模式	28
4.6.4 受控命令桥接模式	29
4.6.5 模拟运行环境回退模式	30
4.6.6 图表化表达与设计约束	30
4.7 本章小结	31
第 5 章 系统实现与运行效果	33
5.1 后端服务与应用入口实现	33
5.1.1 配置管理与应用启动	33
5.1.2 FastAPI 路由与服务组织	33
5.1.3 模型服务与运行时装配	33
5.1.4 主系统流程与助手接入	33
5.2 知识库构建与检索问答实现	37
5.2.1 知识库资料整理与构建	37
5.2.2 文档分块与索引持久化	37
5.2.3 知识库维护与更新机制	37
5.2.4 参数问答与来源展示	39
5.3 意图识别与结构化命令生成实现	40
5.4 DAC-3D 适配与结果解析实现	42
5.5 网页前端交互界面实现	44
5.6 典型运行效果展示	45
5.6.1 关键流程的运行说明	46
5.6.2 运行效果与功能闭环	46
5.7 本章小结	47
第 6 章 系统测试与结果分析	48

6.1 测试目标与测试环境	48
6.2 功能测试结果	49
6.3 接口测试结果	49
6.4 异常与边界测试结果	50
6.5 性能测试与响应指标分析	51
6.6 测试用例覆盖与用户体验评估	51
6.7 场景化验证结果	52
6.8 测试结果分析	55
6.8.1 覆盖性与有效性分析	55
6.8.2 适用范围与改进方向	55
6.9 本章小结	56
总结与展望	57
参考文献	61
致谢	63

第 1 章 绪论

1.1 研究背景

1.1.1 工业检测软件交互背景

人工智能的发展使得算力中心对光模块、光通信的需求逐渐加深，同时伴随精密光学检测、智能制造以及工业软件的持续发展，检测系统所选用的人机交互方式也在逐步发生变化。以往较为常见的交互形式主要依靠按钮式以及菜单式操作，用户通常需要借助固定页面以及既定流程来完成相关任务；而当前更加重视交互过程本身的灵活性，同时也会进一步关注信息反馈是否及时、表达内容是否便于理解。DAC-3D 缺陷检测系统本身面向专业检测任务，系统当中涉及的参数数量较多，操作步骤也相对较长，界面中的状态信息分布较为分散，检测结果中的不少字段还具有一定专业性。因此，在实际使用过程当中，操作人员往往并不能只靠简单点击几次按钮就完成全部工作，而是需要一边查看界面，一边理解参数含义，有时还需要对流程以及结果进行反复确认。

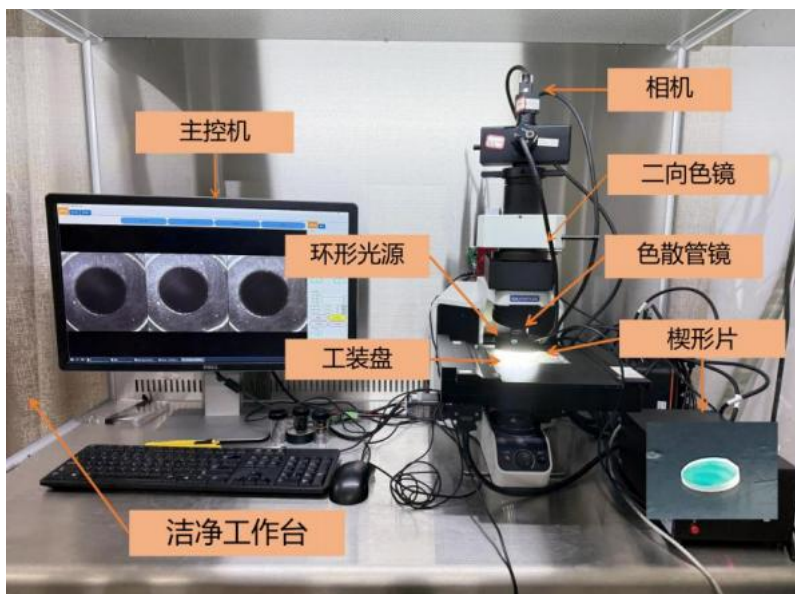


图 1-1 DAC-3D 系统整体结构实物图

1.1.2 DAC-3D 缺陷检测系统使用痛点

从实际应用过程当中来看，DAC-3D 缺陷检测系统所暴露出来的许多问题，并不完全来自设备层面，更多是因为软件操作流程不够直观，以及用户在信息理解方面存在偏差而产生。操作人员在依次完成样品选择、区域确认、参数设置、扫描执行、状态查看以及结果判断等步骤时，如果其中任何一个环节理解得不够清楚，那么后续操作就很容易随之受到影响。比如，扫描区域如果设置得过大，就会使检测时间明显延长；分辨率选择如果不够适宜，也会影响检测结果细节的呈现；结果字段含义如果解释不清，还可能影响后续是

否复检以及是否需要重新调整参数。也就是说，这类系统虽然拥有较为完整的功能，但真正上手并不轻松，新用户通常仍然需要依赖手册、培训，或者向熟练人员请教，因此学习成本一直不低。

传统工业软件的界面大多仍然围绕菜单、按钮、输入框以及状态栏来进行组织，这类交互方式对熟练用户所带来的影响相对有限，因为他们已经理解了相关术语背后所对应的含义，同时也清楚下一步应当开展的具体操作；但对于刚开始接触这个系统的用户来说，许多信息并不能被直接、直观地理解。用户在看到某个参数名称时，并不一定知道它实际控制的具体内容；在看到某个结果字段时，也不一定清楚它与缺陷严重程度之间存在怎样的关系。如果遇到异常情况，软件本身能够提供的帮助又比较有限，很多时候仍然需要返回手册当中进行查阅，或者向有经验的人员进行咨询。用户真正需要的是希望在当前界面当中就可以直接提出“这个参数是什么意思”“为什么这个结果被判成严重”“接下来应该怎么处理”这类问题，并且获得能够理解、也能够继续用于后续操作的回答。

近年来，大语言模型保持了较快的发展态势，并且已经在自然语言理解、知识问答以及内容生成等方面展现出较强能力。尤其是在能够获得外部知识支撑的情况下，模型所生成的回答，相比单纯依靠自由生成会更加稳定，也更容易和具体资料建立起对应关系。对于 DAC-3D 缺陷检测系统这类专业检测软件来说，这种能力具有比较现实的应用意义，因为它可以把手册内容、参数说明、故障处理经验以及结果判定规则整合到一起，使用户能够运用自然语言提出问题，再由系统对相关内容进行解释、提示以及辅助说明。不过，工业检测场景与普通聊天场景之间仍然存在明显差异：检测系统中的回答不能脱离手册和规则去随意发挥，操作请求也不能只依靠文本生成来进行猜测，结果解释更不能只是给出一句模糊判断，而是需要与结果字段、阈值依据以及当前状态相互对应。因此，面向 DAC-3D 构建的智能助手，不能被简单理解为“接入一个聊天框”，而更适宜被看作一种受到知识、规则以及任务边界共同约束的辅助交互层。

在上述背景之下，本文把 DAC-3D 检测场景作为研究对象来开展相关研究，围绕参数问答、自然语言操作、结果解读、状态查询以及操作指导等几类较为典型的应用需求，设计并实现了一个以大语言模型为基础的智能交互助手原型。该课题一方面具有较为直接的应用意义，因为它对应的是实际软件使用过程中培训成本以及理解成本偏高的问题；另一方面也说明，大语言模型在工业软件当中并非只能承担通用对话功能，还可以在明确任务边界的约束下，进一步承担知识解释、流程辅助以及操作支持等更加具体的工作。

1.2 国内外研究现状

1.2.1 大语言模型与检索增强研究现状

在国外相关研究方面，大语言模型的持续演进，为自然语言理解、文本生成以及复杂任务交互等工作奠定了较为重要的基础。Vaswani 等提出 Transformer 架构以后，注意力机

制逐步成为开展长文本建模以及上下文表示的重要方法^[1]；Devlin 等提出的 BERT 也进一步说明，预训练语言模型在语义理解、分类以及信息抽取任务当中具有实际有效性^[2]。在专业知识问答场景当中，如果只是依靠模型参数记忆来生成回答，就容易出现依据不够清楚以及事实存在偏差的问题，因此 Lewis 等提出的检索增强生成方法，把外部检索与生成模型结合起来，使模型可以在回答之前先获取相关知识片段^[3]；Karpukhin 等有关于稠密段落检索的研究说明，向量检索能够有效支撑开放域问答任务^[4]；Sentence-BERT 则为句向量相似度计算提供了常用实现思路^[5]。

在国内相关研究方面，研究人员也已经逐步把大语言模型以及检索增强方法引入到专业应用场景当中。朱卫东等面向电力设备选型设计任务，构建了以向量检索增强大语言模型为基础的智能设计支持系统；其研究结果表明，在专业工程知识库提供支撑的条件下，借助向量检索以及多轮对话机制，可以使复杂选型场景中的知识获取效率得到进一步提升^[6]。余传明以及李昊轩围绕混合任务场景提出了动态检索增强生成框架，并且强调不同任务需要匹配相应的外部知识来源，这与本文把 DAC-3D 用户请求划分为问答、操作、状态以及解释等链路的思路基本保持一致^[7]。

自 2024 年以来，RAG 相关研究已经进一步从基础问答场景，逐步扩展到评估框架、自适应检索以及垂直领域应用等方向。RAGAS 以及 ARES 分别从上下文相关性、答案忠实性以及答案相关性等维度出发，提出了自动化评估方法^{[8][9]}；Self-RAG、CRAG 以及 GraphRAG 则围绕自反思检索、检索纠错以及图结构索引，对高级 RAG 技术开展了探索^{[10][11][12]}。在领域应用方面，医学 RAG 基准、工业知识库 RAG 以及任务型对话检索增强等研究也表明，RAG 在专业场景当中需要结合领域资料、检索质量控制以及任务链路设计来开展应用^{[13][14][15]}。这些研究为本文运用“本地知识库+任务路由+LLM 生成”的系统实现方式，提供了近年的相关研究背景。

1.2.2 任务型对话与工业数字助手研究

任务型对话研究所强调的核心内容在于，系统并不是单纯生成自然语言回答，还需要对用户意图开展识别，对任务状态进行持续维护，并且在这个基础上提供可以执行或能够验证的响应。Yao 等提出的 ReAct 方法展示了语言模型在推理过程以及行动过程之间进行切换的可能性^[16]；Qin 等有关于任务型对话系统开展的综述也指出，任务型系统需要借助外部知识、状态跟踪以及动作选择来形成任务闭环^[17]。国内李帅鹏等提出了显式知识注入的任务型对话理解模型，把自然语言知识插入对话理解过程当中，从而使特定领域中的意图识别以及槽填充能力得到进一步的提升^[18]。上述研究表明，领域知识以及任务理解之间的结合，正是专业软件助手区别于通用聊天系统的关键所在。

从工业应用层面来看，数字助手通常会围绕知识查询、流程引导、异常处理以及辅助决策等任务来发挥作用，并且借助现场信息与业务流程之间的衔接，对操作理解成本进行降低。Zheng 等对生产以及物流场景中的数字助手进行了综述，同时指出工业数字助手的

价值主要体现在知识支持、流程协调以及现场信息获取等方面^[19]。可解释人工智能在视觉检测中的相关研究也表明，如果检测结果缺少缘由说明以及依据表达，那么操作人员就难以对它进行直接理解，也难以形成充分信任^[20]。因此，面向 DAC-3D 这类检测软件所构建的智能助手，不应停留在通用问答层面，而应同时对参数解释、结构化操作、运行状态以及结果依据等内容去进行处理。

1.2.3 研究评述与本文定位

综合来看，现有研究已经在大语言模型、检索增强、任务型对话以及 Agent 工具调用等方向形成了较好的基础，但当把这些方法应用到 DAC-3D 这类具体检测软件当中时，仍然需要进一步处理三类问题。其一，回答内容必须受到设备文档、参数规则以及检测结果的共同约束，否则就容易生成脱离现场语境的泛化解释；其二，操作类请求不宜由模型直接执行，而应当具备命令预览、策略校验、显式确认以及审计追踪等过程；其三，记忆、技能以及检索文档虽然可以对连续交互能力进行提高，但它们本身不能替代或覆盖系统安全策略。

本文的研究工作便围绕上述问题来开展。系统借助本地知识库以及混合检索机制，使回答依据的可追溯性得到进一步提升；依托统一 Agent 入口以及上下文工程，完成不同任务之间的分流以及上下文选择；凭借 Tool Gateway、PolicyEngine 以及 SafetyGuard 建立命令执行边界；同时借助 Memory OS 以及技能系统，补充跨轮交互与流程复用方面的能力。由此可以看出，本文关注的重点并不是提出新的大模型算法，而是面向 DAC-3D 检测业务，构建一套能够运行、可以解释且便于审计的 Agent 化交互原型。

1.3 研究目的与意义

本文的研究目的，是围绕 DAC-3D 检测软件在实际使用过程当中的交互需求，设计并实现一套以大语言模型、检索增强生成以及 Agent 运行时为基础的智能交互助手。该助手并非面向任意问题的通用聊天系统，而是在 DAC-3D 知识资料、运行状态、结构化命令、安全策略以及检测业务边界等共同约束之下开展工作的任务型辅助模块。系统需要拥有参数问答、流程说明、自然语言命令预览、运行状态查询、检测结果解释、命令确认、记忆审核以及评测追踪等方面的功能。

围绕上述研究目标，本文主要对以下四个问题进行解决。第一，如何把 DAC-3D 手册、参数说明、常见问题以及缺陷规则组织成可检索的知识内容，从而使系统回答拥有明确依据。第二，如何在多轮交互过程当中，选择当前任务实际需要的运行状态、技能说明、记忆以及安全策略，以避免上下文出现无序堆叠。第三，如何把自然语言操作请求转化为可以校验、可以确认且可以追踪的结构化命令预览，进而避免模型直接驱动检测流程。第四，如何把工具调用、命令确认、记忆写入以及评测结果纳入可审计闭环。

本文的主要贡献体现在四个方面。第一，构建了面向 DAC-3D 场景的知识约束型问答

以及操作辅助框架，使回答内容与操作提示可以依托场景知识来展开。第二，设计了统一 Agent 运行时以及上下文工程机制，使问答、状态、结果、控制、记忆以及技能任务能够在同一入口下进行分流处理。第三，设计并且实现了受控 Tool Gateway 和安全审查链路，使执行类命令必须经过预览、校验、确认以及审计。第四，补充了 Memory OS、技能系统以及 Trace/Eval 闭环，使系统具备跨轮交互、流程复用以及回归验证能力。

从应用意义方面来看，本研究可以在一定程度上对 DAC-3D 软件的学习成本以及操作理解成本进行降低，使参数解释、流程查询、状态确认以及结果分析等环节能够借助自然语言来完成，这样一来更加契合用户的交互习惯。从工程意义方面来看，本文将大语言模型能力与本地知识库、任务识别、结构化数据以及主系统适配层进行结合，为大语言模型在专业工业软件当中的可控集成给出了较为具体的实现案例。

1.4 研究内容与论文结构

本文按照“问题分析—方法设计—系统实现—测试讨论”的逻辑，对研究内容进行组织。本文的研究重点并不是对 DAC-3D 的图像检测算法、点云处理流程或硬件控制策略开展重新设计，而是在既有检测软件之外，构建一个能够独立说明、接受验证并且支持审计的 Agent 化交互层。该交互层需要把知识问答、状态读取、结果解释、命令预览、安全确认、记忆审核以及评测追踪组织成完整链路，进而回答大语言模型如何在专业工业软件当中得以受控使用这一问题。

在方法与实现部分，本文围绕统一 Agent 入口、Context Builder、SkillRegistry、Memory OS、Tool Gateway、SafetyGuard、DAC-3D 适配以及 React 前端展示来开展设计工作。统一 Agent 入口负责把用户请求分派到问答、状态、结果、控制、记忆或安全等不同链路；Context Builder 负责选择本轮交互当中真正需要的文档、状态、技能以及记忆；Tool Gateway 和 SafetyGuard 则负责把执行类命令约束在预览、校验、确认以及审计流程之内。借助这种组织方式，论文各章不再只是停留在模块罗列层面，而是围绕同一个工程问题，按照功能依赖关系逐层展开。

本文的章节结构安排如下。第 1 章主要对研究背景、研究现状、研究目的以及论文结构进行介绍，同时进一步明确本文所涉及的系统边界。第 2 章围绕大语言模型、检索增强生成、Agent 运行时、工具调用安全以及 DAC-3D 桥接等相关技术开展说明。第 3 章从业务流程、用户角色、功能需求以及非功能需求等方面对问题进行界定。第 4 章提出系统总体设计以及核心数据流。第 5 章对关键模块的实现过程进行说明。第 6 章按照验证目标、测试环境、评价指标、运行结果、失败案例以及适用范围，对系统进行分析。第 7 章总结本文工作，并给出不足之处与后续展望。

第 2 章 相关技术与理论基础

该原型一方面应具备对参数说明与流程咨询进行回答的功能，另一方面还需要对自然语言操作请求开展预处理，同时支持检测结果解释以及运行状态查询。因此，本章的分析重点并不局限于某一单独算法，而是进一步考察上述技术在 DAC-3D 场景中的协同关系以及作用方式，从而形成具备工程落地条件的系统方案。

2.1 大语言模型与任务型交互

大语言模型是以大规模语料预训练为基础开展建模的一类自然语言处理模型，可以承担文本理解、语义归纳、上下文组织以及自然语言生成等方面的任务。它的核心技术基础一般包括注意力机制、预训练表示、指令对齐以及上下文学习等内容。对于工业检测软件而言，大语言模型的作用并不是去替代既有检测算法，而是对用户借助自然语言提出的参数问题、操作意图以及结果疑问进行理解，并且把这些表达进一步转换为更加清晰、能够用于系统交互的响应内容。

任务型交互和开放式聊天之间具有比较明确的差异。开放式聊天主要关注回答内容是否流畅以及对话上下文是否连续，而任务型交互则更强调对意图识别、任务状态维护、外部工具或知识调用以及结果约束等环节进行组织，并且需要把这些环节统一纳入同一任务流程当中。在本文所面向的应用场景中，用户输入可能分别指向参数问答、扫描操作、状态查询、结果解释或故障指导等不同任务。如果系统未先对任务类型开展区分，而是把全部输入直接送入同一生成链路去处理，那么就容易出现操作请求被当作普通问答、结果解释脱离检测字段、状态查询无法读取运行信息等问题。

因此，本文将大语言模型界定为 Agent 运行时当中提供语义理解、任务分派以及文本组织能力的能力来源，而并非直接控制 DAC-3D 主系统执行过程的核心组件。系统首先会对用户请求类型进行判断，然后结合知识库、技能说明、运行状态或结果字段来生成相应回复。对于参数问答，模型主要依据检索资料组织解释内容；对于操作请求，模型仅参与命令预览的形成，后续执行还必须经过 Tool Gateway 校验以及用户确认；对于结果解释，模型只围绕结构化结果以及阈值规则生成说明。相关内容归纳见表 2-1。

表 2-1 大语言模型在任务型交互过程当中的功能作用以及约束条件

能力	在本文中的作用	约束方式
语义理解	对用户问题当中所包含的参数、流程、状态、结果、记忆以及控制意图开展统一的识别与分类工作	首先，会由统一 Agent 入口对任务开展分派工作，随后再进入相应的受控处理链路当中。
文本生成	把检索资料、结构化字段、技能说明以及工具返回结果，统一整理成自然语言回答	回答内容应当把资料来源、相关字段以及阈值依据结合起来，对结论形成的过程进行说明。

续表 2-1 大语言模型在任务型交互过程当中的功能作用以及约束条件

能力	在本文中的作用	约束方式
工具调用	对 DAC-3D 状态开展读取工作，生成命令预览内容，对命令进行校验，提交已经确认的命令，同时读取 Trace 记录	必须通过 Tool Gateway、PolicyEngine、SafetyGuard 和 schema 校验。
上下文组织	从运行状态、记忆、技能以及检索资料当中，进一步筛选本轮交互实际所需的内容	Context Builder 主要负责对上下文开展压缩、分级以及隔离处理，未经审核的内容不得被授予执行权限。
安全审查	对高风险命令、路径越权、确认绕过以及 Prompt Injection 等风险开展统一识别工作	针对高风险命令，系统应当要求用户进行显式确认；针对未知工具以及违规请求，则需要在默认情况下予以拒绝。

2.2 检索增强生成与混合检索

本文之所以在系统当中引入大语言模型，主要缘由在于它在自然语言理解、上下文组织以及文本生成等方面具有较强的支撑能力。对于 DAC-3D 这类专业检测软件来说，用户在实际提问时往往不会严格选用参数名称或标准术语，而是会借助相对口语化的说法来描述参数含义、故障现象以及检测结果。要是完全依靠固定表单或关键词匹配，系统就较难覆盖这类表达不够规范的输入；而在检索增强以及结构化约束共同配合的情况下，大语言模型可以把用户表达转换为更为清晰的问答内容、操作指导或任务处理结果。

不过，本文并没有把大语言模型当作一个可以独立解决全部问题的模块来使用。从 DAC-3D 这类专业应用场景来看，如果缺少必要的知识约束，模型虽然能够生成形式上较为完整的回答，但也容易出现事实准确性不足、内容范围扩展过多以及回答依据不够清楚等问题。参数说明、操作指导以及结果判断等任务本身具有较强的专业属性，系统输出一旦脱离用户手册、规则体系以及结构化结果，就可能影响用户对相关信息的理解以及后续判断。鉴于这一考虑，本文并未让模型进行自由式回答，而是把检索增强生成方式一并引入系统响应过程当中。

检索增强生成的基本思路，是先把领域资料整理成能够被系统检索的知识库；在用户发起请求之后，系统会对与问题有关的文本片段进行检索，并且把检索结果、用户问题以及任务提示一并送入生成模型^{[3][21]}。本文构建的知识库资料主要包括 DAC-3D 操作手册、参数说明、常见问题以及缺陷规则，并且在完成文本清洗、分块、向量化以及索引持久化等工作后，形成可供后续调用的文档集合^{[5][22][23]}。经过上述处理，模型输出就不再只是依靠参数记忆，而是凭借检索片段以及任务约束，对回答内容进行组织。

对于本文所设计的系统来说，这种处理方式具有几个较为直接的作用：回答内容可以更容易地同已有资料保持一致；在开展知识补充工作时，也不需要对模型进行反复训练；

前端还可以把来源信息同步展示出来，这样便于用户在后续使用过程中进行核对。也正是鉴于这一考虑，本文将参数问答、操作指导以及结果解释等任务尽量纳入“检索结果 + 任务提示 + 模型生成”的处理模式当中，使不同类型任务可以在知识约束层面保持基本一致。

系统在模型调用环节同时保留了本地离线回退以及外部 API 调用两条路径。项目中围绕模型接入工作设置了本地模型回退模式以及兼容式模型服务等适配方式。前者主要用于本地测试、自动化验证以及无网络条件下的原型开发工作，后者则面向后续真实大语言模型服务的接入。借助这种双路径设计，系统既可以在当前条件下保持较为稳定的运行状态，也能够为后续真实模型接入预留必要的扩展空间。本文选用这一技术路线，并不是单纯追求模型能力本身，而是更加关注知识约束、系统可用性以及原型完成度之间的平衡关系。

2.3 意图识别与结构化命令生成

在 DAC-3D 智能交互助手当中，用户输入不适宜被统一纳入普通问答流程来进行处理。如果系统把不同类型的内容都归入简单的自然语言问答，那么后续执行链路就会变得较难约束，尤其是在操作请求以及结果解释这两类场景中，系统若只返回一段自然语言文本，往往难以充分契合软件实际使用过程中的要求。因此，系统需要首先对用户当前意图进行判断，明确用户究竟是在询问参数、请求执行操作、解释检测结果、获取处理建议，还是查看当前状态；只有在任务类型被明确之后，后续模块才可以进入对应的处理路径，并且开展后续处理工作。

鉴于这一考虑，本文把用户输入划分为问答、操作、结果解释、操作指导以及状态查询五类意图。其中，问答意图主要对应参数说明以及流程咨询类问题，操作意图对应扫描类操作请求，结果解释意图用于对检测结果进行解读，操作指导意图对应现场处理建议，状态查询意图则面向运行状态读取。这样的划分并不是为了让标签设置变得更加细碎，而是为了使不同类型请求可以进入各自适配的处理链路。例如，“扫描 25mm×25mm 区域”和“25mm×25mm 区域是什么意思”两句话，表面上都包含较为接近的数字表达，但前者在本质上属于操作请求，后者则更接近参数或流程问答。如果系统没有先对意图开展区分，那么后续处理过程就容易出现路径偏差。

在上述几类意图当中，操作类请求对系统约束会提出更高要求，因此，结构化命令生成以及受控工具调用在本文中具有较为关键的作用。工业软件中的很多参数通常都对应明确的取值范围以及使用条件，不能只是因为用户给出一句自然语言描述，系统就马上把它转换为可执行操作。本文选用的处理方式是，先由 Agent 开展任务分派，再形成结构化命令预览，随后交由 Tool Gateway 对 schema 校验、路径白名单检查、风险分类、确认令牌校验以及命令生命周期记录进行统一处理，而不是直接触发执行。这样一来虽然增加了必要的中间处理环节，但更契合工业软件对于安全性以及可控性的基本要求。

例如，当用户输入“我想扫描一个 25mm×25mm 的楔形滤光片，步长 10 微米”时，系

统会先对扫描区域以及步长等信息开展抽取工作，并把这些内容整理成结构化对象结果；如果用户只是输入“开始扫描”，却没有给出区域尺寸、分辨率或模式信息，系统就不会自行推断并补齐这些关键参数，而是会返回澄清提示，引导用户继续补充必要内容。因此，系统输出不仅可以形成可校验的结构化对象，还能够进入后续受控处理链路，并且在关键信息不完整时及时停止处理，而不是继续向下执行。

结构化命令生成还具有较为直接的工程作用，即便于前端与后端之间开展协同，并且为后续运行时接入预留必要的处理接口。自然语言说明更适宜面向用户进行阅读，结构化对象结果则更适宜用于程序校验、界面展示以及后续提交处理。鉴于这一点，本文在命令链路当中同时保留结构化字段以及 YAML 文本预览，主要是为了兼顾上述两类使用方式。经过这样的处理，前端能够更加清楚地展示关键参数；如果后续继续接入真实 DAC-3D 运行时，结构化对象结果也可以较为自然地转交给适配层去处理。因此，该模块并不是简单把一句话拆分为若干字段，而是推动系统从普通问答进一步转向受约束的任务处理过程。

2.4 DAC-3D 主系统桥接与结果解析技术

为了能够实现界面层、Agent 编排层以及底层设备协议之间的有效解耦，本文将 DAC-3D 的访问边界单独收束到适配模块以及 Tool Gateway 当中进行管理。当前系统同时拥有模拟运行环境、状态文件桥接以及受控命令桥接三类能力：其中，模拟运行环境主要面向本地演示以及测试工作，状态文件桥接用于读取 DAC-3D 主窗口在开始检测、点位结果更新、检测完成或停止时所写出的 JSON 状态信息，受控命令桥接则只接收已经经过预览、校验以及确认流程的结构化命令。经过上述处理之后，前端无需直接接触底层通信细节，Agent 也不能直接写入命令文件，系统整体层次因此会更加清晰。

本文之所以将 DAC-3D 适配层以及工具网关单独拆分出来，主要缘由在于界面展示、问答逻辑、Agent 推理以及设备通信并不适宜被混合放在同一条处理链路之中。如果前端或模型直接依赖底层协议，那么一旦后续设备通信方式发生变化，系统其他部分也会被迫同步开展调整；如果 Agent 能够绕过网关直接写入命令文件，那么系统安全边界就较难在代码层面得到保障。为了能够避免上述问题，本文把 DAC-3D 访问能力集中封装在统一适配模块以及 Tool Gateway 当中，使外部模块只需要依赖稳定接口，就可以完成状态获取、命令预览生成、确认提交以及结果读取等操作。

状态文件桥接在本文原型中仍被定位为本地真实状态的接入方案，不过它已经不再承担完整控制边界。具体来看，DAC-3D 主窗口会在开始检测、各点位结果更新、检测完成或停止等节点写出 JSON 状态文件，助手侧则会在系统启动阶段，或者接收到状态查询请求时读取该文件，并把它作为实时状态信息来源。对于可能改变运行状态的命令，系统会先在助手侧生成 pending preview，并且在经过 PolicyEngine、SafetyGuard、PathPolicy 以及确认 token 处理之后，才允许受控桥接层继续提交。该方式既保持了状态读取过程的低侵入性，也避免了大语言模型直接写入命令文件带来的风险。

在结果解释环节当中，系统并不会把原始结果直接返回给用户。原始检测结果通常包括深度值、长度值、位置坐标、规则编号以及状态字典等内容，这些数据虽然有利于程序继续开展后续处理工作，但对于普通操作人员来说并不够直观，也较难直接支撑现场判断。因此，本文选用的处理方式，是先借助结果解析模块对原始结果进行整理，并且将它归并为缺陷类型、严重度、位置、测量值、阈值说明以及判定原因等统一字段，随后再结合缺陷规则文档生成解释文本。经过这样的处理之后，系统可以把“原始结果是什么”“规则如何判定”以及“最终应如何说明”划分为几个比较清楚的层次。

以当前原型中的划痕缺陷为例，系统会依据深度、长度等阈值规则开展判定工作，并给出 low、medium 以及 high 三个等级的严重度结果，同时保留触发该判定过程的关键字段。相较于直接生成一整段说明文本，先进行解析、再进行解释的处理方式更适宜工业检测场景，因为它会先把检测结果整理成较为稳定的结构化内容，再围绕这些字段组织语言说明，从而降低解释文本与原始字段之间出现不一致的可能性。也就是说，本文并未选用模型直接读取结果后自由生成解释的方式，而是选用了先适配、后解析、再解释的分层处理链路。

这种分层安排与软件工程中较为常见的模块化组织思路基本契合。适配层主要承担通信过程封装工作，解析层主要承担数据结构整理工作，解释层则负责把结构化结果进一步转换为面向用户的说明性表达。三者完成相对分离之后，不仅便于分别开展测试工作，后续在进行替换以及维护时也会更加清楚。例如，在接入真实 DAC-3DSDK 时，主要改动通常集中在适配层；如果缺陷规则后续发生变化，主要调整会落在解析逻辑当中；如果前端界面进行调整，前两层一般不需要随之出现较大范围改动。正是依靠这种拆分方式，本文形成的并不只是单纯的功能演示，而是一套具备继续扩展条件的软件原型。

本文选用前后端分离架构，并不是只从界面展示这一方面进行考虑，更主要是为了契合当前由 DAC-3D 主系统跳转至网页端助手界面的集成方式，同时也为后续更紧密的客户端集成预留必要空间。如果把全部逻辑都放入单体界面当中，那么后续无论是对前端样式进行调整、接入新的接口，还是更换模型服务，都会造成不同模块之间的相互牵连。当前原型的接口组织借鉴了网络化软件架构以及软件架构设计中的分层、解耦思想^{[24][25]}，后端接口服务选用 FastAPI^[26]，前端主界面选用 React^[27]，知识库的存储组件则选用 Chroma^[28]。同时，项目还保留了 Gradio 兼容界面，用于开展快速调试工作^[29]，运用自动化测试框架来组织回归测试^[30]，并且借助 Server-Sent Events 支持流式输出。上述技术共同构成了本文原型的工程基础。

从当前实现情况来看，后端主要对外提供聊天接口、流式聊天接口、运行状态接口以及有关于知识库管理的相关接口，用来支撑聊天问答、流式输出、状态查询以及资料重建等方面的能力；前端则把网页端对话界面当作主要入口来使用，负责进行多轮消息展示、主题切换以及消息负载承载等基础交互工作^[27]。DAC-3D 主系统可以借助入口按钮、菜单或者页面跳转等方式打开该网页界面，进而调用问答、命令预览、结果解释以及状态查询

等功能。在完成上述划分之后，如果后续需要对界面风格进行调整，那么相关改动通常会集中在前端；如果后续需要增加新的处理逻辑，那么改动则主要集中在后端，这样一来就不必在每次修改时牵动整套程序。

在交互方式的设计方面，本文没有继续沿用传统表单式工业软件界面，而是把对话框当作主要入口来使用；这种设计并不是放弃结构化约束，而是把“入口自然语言化”以及“内部处理结构化”结合起来。用户可以先借助普通语言对具体需求进行表达，系统随后会依据任务类型返回命令预览、来源依据、结构化结果以及知识库状态。为了能够对交互过程的可见性进行提高，前端还选用了流式输出方式，使回答可以按照文本块逐步显示，这既契合当前主流智能助手产品的交互习惯，也更有利于后续展示程序运行状态以及检测结果。

同时，本文也没有把前端简单地当作文本显示窗口来使用，而是依据返回内容的不同类型，对对应展示方式进行了划分以及安排。普通回答会进入聊天区域，来源依据、结构化命令以及状态摘要则会被放置到详情区域或设置区域当中进行展示。这样一来，用户在一轮交互过程里不仅可以查看系统所生成的回答内容，还可以进一步核对系统依据了哪些资料、抽取了哪些字段，以及当前运行状态处于什么情况。对于 DAC-3D 这类专业软件来说，这种展示方式相较于把所有内容直接堆叠在对话框中，会更便于用户阅读，也更接近实际辅助工具的使用方式。

此外，当前项目已经把网页端确立为主要的交互入口来使用，兼容界面则仅作为调试以及旧环境验证的保留方案，不再被纳入论文所讨论的核心技术路线当中。Web API 层还进一步增加了命令审批、Trace 读取、评测运行、评测草稿生成、目标管理以及记忆补丁审核等接口，使前端不再只是承担聊天内容展示功能，也可以对 Agent 工作台所需要的安全信息以及运维信息进行呈现。

2.5 FastAPI、React 与前后端分离架构

本文原型以前后端分离方式作为整体构建基础：后端接口服务主要承担聊天问答、状态查询、知识库摘要、流式输出以及文档上传重建等方面的能力，前端页面则负责开展对话消息、来源依据、结构化命令以及状态面板的组织与展示工作，前端构建工具用来支撑页面开发以及打包流程。借助这一技术组合，任务处理逻辑与界面展示逻辑可以保持相对独立，这样一来，也为后续嵌入 DAC-3D 主系统，或者进行单独部署提供了必要条件。

FastAPI 较为适宜作为本文后端接口层来使用，其主要缘由在于，它只需要较少的框架配置，就可以把请求校验、路由分发以及响应模型进行统一组织，同时还便于把聊天、运行状态、知识库摘要、知识库重建以及流式输出等能力拆分成边界比较清晰的接口。对于本文这类原型系统来说，后端接口既需要服务于网页端，也需要为后续嵌入 DAC-3D 主系统保留可能性，因此接口层应尽量保持轻量以及清晰，并且与知识库、意图识别、运行时适配等业务模块维持明确边界。

React 以及 Vite 则更适宜用来承担本文的前端展示层工作。本文前端页面需要同时展示对话内容、来源依据、结构化命令、运行状态以及知识库信息，而这些内容在界面更新以及交互反馈方面，都带有较为明显的状态驱动特性。借助组件化的前端组织方式，可以把聊天消息、来源列表、命令预览、状态面板以及知识库摘要分别整理成相对独立的展示单元；后续如果需要对某一部分的显示方式进行调整，也通常不会直接牵动后端的处理逻辑。Vite 的主要作用在于对前端开发以及构建过程中的成本进行降低，使原型界面可以较快地完成迭代。

从系统集成这一方面来看，前后端分离同样有较为明确的现实意义：DAC-3D 主系统本身已经承担了相机、图像处理、数据库以及界面交互等较重任务，如果继续把模型调用、向量检索以及复杂交互界面直接纳入主程序当中，不仅会增加主系统在运行时的负担，还会进一步提高后续开展调试工作的难度。因此，把助手能力以独立服务的形式进行部署，并且借助网页入口、状态文件或嵌入式桥接方式与主系统建立连接，会更加契合低侵入式集成原则。

2.6 本章小结

本章主要围绕 DAC-3D 智能交互助手原型当中所涉及的关键技术以及具体场景需求来展开分析，并且分别从需求分类、大语言模型与检索增强生成、意图识别与结构化命令生成、DAC-3D 适配与结果解析，以及前后端分离交互界面等方面进行了说明。借助上述分析可以看出，本文所构建的系统并不是单纯为原有检测软件额外增加一个聊天入口，而是希望在知识约束、任务分流、结构化输出以及运行时适配等多个层面，逐步形成较为完整的辅助交互能力。

同时，本章的相关分析还进一步说明，在 DAC-3D 场景当中，系统实现的难点并不只是表现为单一模型能力能否得到进一步提升，而是更多地落在如何把知识内容、业务规则、运行状态以及交互界面组织成一套可以运行、能够验证并且具备扩展空间的任務型系统。下一章将在此基础上继续开展总体设计方面的说明，并且对系统架构、数据结构、功能流程以及接口组织形式进行具体阐述。

第 3 章 系统需求分析

在相关技术基础已经得到明确梳理之后，还需要进一步对 DAC-3D 智能交互助手所面向的真实业务环节、所服务的使用对象，以及各类需求同本项目实现模块之间形成的对应关系进行说明。本章以 DAC-3D 主系统、助手后端、前端界面以及测试用例作为主要依据，分别围绕检测业务流程、用户角色、功能需求、非功能需求以及典型用例这五个方面开展分析工作。

3.1 DAC-3D 检测业务流程分析

3.1.1 检测准备阶段

检测准备阶段主要包括对样品进行放置、对在线或离线模式进行选择、确认离线原图目录、理解扫描区域以及分辨率、调整光照或曝光，并且核对基础参数等工作。在主系统流程当中，PyQt 界面主要负责对运行按钮、离线检测控件、144 点位按钮以及历史统计入口进行组织与触发；助手侧则需要围绕这些真实操作节点，提供参数问答、操作指导以及资料来源展示能力，而不能脱离主系统流程去单独生成泛化建议。

DAC-3D 交互界面如图 3-1 所示。

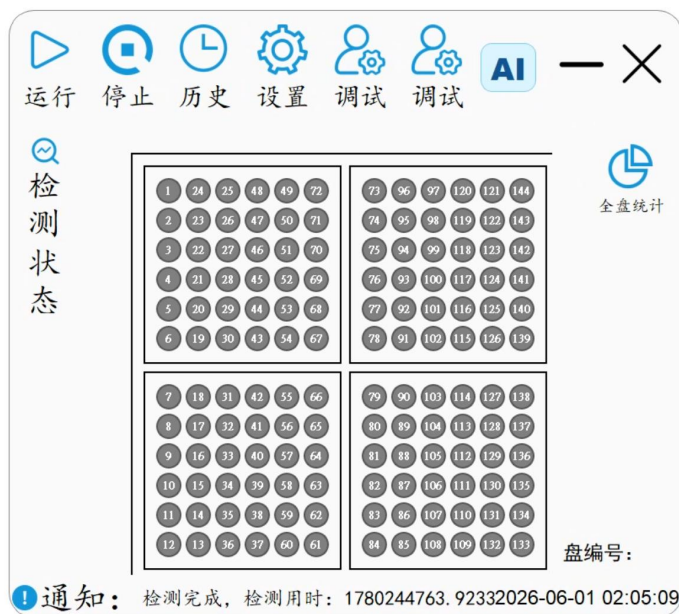


图 3-1 DAC-3D 交互界面示意图

3.1.2 检测执行阶段

在检测执行阶段当中，系统需要进一步强调运行过程所具备的可见性以及操作链路的安全性。本项目的主系统主要由界面进程、调试/硬件控制进程以及图像处理进程共同组成；其中，在线模式会依靠控制队列来触发 144 点位扫描，离线模式则借助图像目录去读取三

相机焦前、焦面、焦后原图，同时凭借两个样品结果对象来生成样品结果。鉴于扫描启动、离线检测以及停止检测都会对真实业务流程产生影响，助手不能把自然语言请求直接看作已经执行完成的结果，而应当先生成结构化命令预览，并且在依次经过 Tool Gateway、策略校验、路径白名单、风险分类以及用户确认之后，才允许进入受控桥接流程。

3.1.3 结果分析阶段

在结果分析阶段当中，系统需要把图像处理进程所输出的检测结果，进一步转化为操作人员能够理解的解释内容。当前主系统通常会先对三相机图像开展配准融合工作，再调用已有缺陷检测模块，对飞溅、划痕、点蚀等瑕疵进行识别，并且依据不同检测区域、尺寸、忽略规则以及合格性字段，生成结果图、明细文件、统计文件以及数据库记录。上述检测模型以及检测流程均属于 DAC-3D 主系统已经具备的既有能力，本课题不涉及检测模型训练、改造以及部署方面的工作。助手侧需要读取最近结果字段、结果历史字段，或者嵌入式运行时返回的数据，并在已有检测结果基础上完成字段解析、严重度说明以及自然语言解释工作。

相关内容归纳见表 3-1。

表 3-1 DAC-3D 检测流程与助手介入节点表

流程阶段	主要任务	用户痛点	助手介入方式
检测准备	先对样品进行固定处理,随后在此基础上完成在线/离线模式选择以及目录、参数的确认工作	对于参数含义、目录要求以及操作前提等方面的内容,用户仍然需要反复开展查阅以及确认工作。	目前系统已经具备了参数问答、离线目录说明、操作指导以及来源依据展示等方面的功能。
检测执行	在在线状态下开展 144 点位扫描任务,在离线模式下对三相机图像进行读取,同时完成检测停止操作	自然语言请求通常并不容易被直接转化为可以安全执行的操作,而运行状态又分散地呈现在主系统界面以及进程队列当中	生成结构化命令,并且提示用户对其中缺失的字段进行补充,同时对状态文件或嵌入式桥接状态开展读取工作
结果分析	对图像融合、瑕疵检测、区域判定、结果文件以及数据库记录输出等环节开展统一组织工作,从而保证处理流程可以连续衔接	原始字段以及区域规则本身缺少面向操作人员的表达方式,因此并不适宜被直接呈现出来供其阅读	对最近结果以及历史结果开展解析工作,并围绕缺陷情况、合格性以及判定缘由进行说明

3.2 用户角色与使用场景分析

3.2.1 操作人员角色

操作人员是系统主要所面向的使用对象，其关注点通常会集中在参数含义理解、扫描请求确认、运行状态查看、缺陷结果解释以及现场故障处理等方面。该角色一般不需要深

入掌握底层模型调用以及向量库的实现机制，但需要能够判断助手回答是否具有资料依据、命令预览是否完整，以及当前状态是否可以支持继续开展下一步操作。

3.2.2 系统维护人员角色

系统维护人员主要负责开展知识资料维护、Agent 运行配置、安全策略验证以及评测用例维护等工作。该角色需要能够查看知识库摘要，触发知识库重建，并且检查接口返回结构；同时，当模型服务、状态文件、记忆补丁、命令审批或前端资源发生变化时，也需要能够及时开展问题定位以及修复工作。

3.2.3 DAC-3D 主系统与助手服务边界

DAC-3D 主系统主要负责实际检测流程的执行，以及三进程队列通信、图像融合、瑕疵检测、结果落库和状态文件写入等工作；助手服务则更侧重于对知识问答、Agent 任务分派、上下文构建、结构化命令预览、状态读取、结果解释、安全审批、记忆审核以及评测追踪等任务进行处理。当前项目当中同时保留了两类系统边界：一类是 Web 助手借助状态文件、命令桥接以及 Tool Gateway，与主系统进行低侵入式交互；另一类是嵌入式助手面板依靠桥接接口来调用主窗口能力。因此，论文中的系统边界说明会围绕这两类已经形成的真实实现来展开，并且明确 LLM 不能直接写入 DAC 命令文件。相关内容归纳见表 3-2。

表 3-2 用户角色与需求说明表

角色	关注内容	主要需求	系统响应
操作人员	参数、状态、结果和操作建议	用自然语言快速获得可理解回答	目前系统已经拥有聊天问答、来源依据展示、命令预览以及结果解释等方面的功能。
系统维护人员	资料、接口、配置和测试结果	维护知识库并验证服务可用性	对知识库摘要、重建接口调用、健康检查以及自动化测试等工作开展统一梳理
DAC-3D 系统	检测状态、命令接收和结果发布	凭借低侵入方式向助手提供必要的运行信息，并且接收经过约束处理后的可控命令	对状态写入流程、命令轮询处理流程，以及嵌入式桥接机制开展统一梳理工作
助手服务	任务路由、知识问答和结果组织	在受约束流程中处理用户请求	对 RAG、意图识别、结构化命令、运行时适配模块以及 FastAPI 接口等内容开展统一梳理工作

3.3 功能需求分析

从项目目前已经完成的实际实现范围来看，系统功能需求可以被整理为参数问答、自然语言操作预览、运行状态查询、检测结果解释、操作指导以及知识库维护这六类内容。

其中，这些需求并不是彼此孤立存在的，而是分别同知识库检索、意图识别、结构化命令生成、运行时适配、结果解析以及网页接口等模块建立了对应关系。

功能需求优先级的划分，同 DAC-3D 在实际使用过程中当中的业务流程之间具有较为直接的对应关系。参数问答以及操作指导主要服务于检测准备阶段，用来降低用户在理解手册内容、参数含义以及流程步骤时所需要承担的成本；自然语言操作预览以及状态查询则主要面向检测执行阶段，可以把用户相对口语化的表达转换成便于核对的中间结果，同时也帮助用户确认当前检测是否正在运行、是否已经完成，或者是否已经出现异常情况；检测结果解释主要面向结果分析阶段，用来把缺陷类型、测量值、检测区域以及阈值依据进一步转化成更容易理解的说明内容。

这些功能需求并不是彼此孤立的条目，而是在业务流程当中具有较为清楚的先后次序以及约束关系。系统只有先对知识库开展检索工作，参数问答以及操作指导才会拥有对应的资料依据；只有先完成意图识别以及字段校验，操作类请求才不会越过必要的安全确认环节；只有先完成运行时适配以及结果解析，状态查询以及结果解释才不会停留在静态文本说明的层面。因此，表 3-3 中的功能需求既构成了用户侧能够感知的功能清单，也是第 4 章开展模块划分以及第 6 章进行测试设计的重要基础。

相关内容归纳见表 3-3。

表 3-3 功能需求分类表

编号	功能需求	输入形式	输出结果	约束说明
F-01	参数问答	参数名、自然语言问题	带来源依据的参数解释	优先依靠知识库资料以及来源元数据作为依据。
F-02	Agent 任务分派	任意用户消息	任务类型、工具选择和上下文摘要	并不会把全部请求都统一放入同一条自由问答链路当中。
F-03	自然语言操作预览	扫描、离线检测或停止检测说明	结构化命令、缺失字段、风险提示以及确认方面的信息	执行前必须进入 Tool Gateway 生命周期。
F-04	运行状态查询	状态、进度或设备问题	状态摘要和最近结果信息	使用只读工具，不触发控制动作。
F-05	记忆与技能辅助	跨轮指代、用户偏好	记忆命中、技能说明	可审核、可拒绝。
F-06	评测与审计	运行 Trace 或评测请求	Trace 记录、回归结果和安全用例结果	敏感字段脱敏，失败用例可追踪。

3.4 非功能需求分析

除了功能完整性之外，工业检测软件助手还需要在准确性、可追溯性、操作安全性、响应效率、可扩展性以及可维护性等方面，满足对应的非功能要求。尤其是当系统涉及扫描请求以及结果解释时，应当优先保证参数信息的完整性以及依据来源的清晰性，不能把生成文本是否流畅当作结果可验证性的替代依据。

非功能需求在本文当中并不是附属层面的要求，而是用来判断系统能否真正进入工业检测场景的关键条件。普通聊天系统通常更关注回答内容是否自然、是否流畅，而 DAC-3D 智能交互助手则需要优先保证答案拥有明确的来源依据，命令已经经过必要确认，状态来自可信入口，并且结果解释可以同结构化字段对应起来。如果缺少这些约束，即使界面能够正常展示回答内容，也不能据此说明系统已经契合专业软件辅助交互的基本要求。

从具体实现层面来看，准确性与可追溯性依靠知识库来源、检索分数、章节信息、结果字段以及 Trace 记录来进行保证；操作安全性则主要借助命令预览、缺失字段校验、Tool Gateway、PolicyEngine、SafetyGuard、路径白名单以及确认令牌来加以体现；低侵入式集成主要依托状态文件、命令桥接以及嵌入式桥接边界得以实现；响应效率以及可维护性，则需要凭借模块拆分、接口契约、持久化索引、上下文压缩以及可重复评测来提供支撑。

相关内容归纳见表 3-4。

表 3-4 非功能需求说明表

需求类别	具体要求	设计约束
准确性与可追溯	在生成回答内容时，应当尽可能把文档片段、结构化结果、工具输出以及 Trace 当作依据来进行说明	对来源、章节、检索分数、字段信息以及 trace_id 等内容进行返回。
操作安全性	如果关键参数存在缺失、路径存在越权或动作具有较高风险，系统不应直接进入执行流程	命令预览、风险分类、确认 token 以及命令生命周期等相关机制，仍然需要被完整保留下来。
上下文可信隔离	检索资料、记忆以及工具输出，都不能对系统安全策略形成覆盖，也不能替代它的作用。	Context Builder 记录 trust_level 以及 can_influence_tools 等边界。
可维护性	知识、技能、记忆、工具以及安全策略，应当按照分层方式开展维护工作	各模块借助结构化对象以及接口来开展协作，从而避免相互之间形成直接耦合。
可验证性	关键链路应当可以借助自动化测试以及本地评测来开展复查工作	Trace/Eval 会对工具调用过程、风险决策结果以及最终输出内容进行记录。
低侵入集成	不会对 DAC-3D 检测算法以及主流程控制边界进行改动	状态文件、命令桥接以及嵌入式桥接，只作为适配边界来存在。

3.5 用例分析

鉴于上述需求，本文选取参数问答、扫描命令预览、离线检测目录校验、状态查询、结果解释以及知识库重建这六类用例来进行说明。这些用例分别对应检测准备、检测执行、结果分析以及系统维护过程当中核心交互环节，可以体现系统在不同业务阶段所承担的功能作用。

相关内容归纳见表 3-5。

表 3-5 典型用例说明表

用例名称	基本流程	异常流程	输出结果
参数问答	当用户输入有关于参数的问题时，系统会先对相关资料开展检索工作，并且把检索结果作为依据来生成回答。	知识库为空时提示先构建资料	参数解释和来源依据
扫描命令预览	在用户输入扫描区域以及分辨率之后，系统会对相关字段开展抽取工作，并且据此生成对应的命令预览内容	如果区域或分辨率字段存在缺失，系统会返回对应的澄清问题，用来补全相关信息	结构化命令对象与文本预览
离线检测目录校验	在用户输入目录路径之后，系统会对图像目录字段开展抽取工作，并且进一步对图像数据的完整性进行检查。	如果目标目录并不存在，或者图像文件有缺失情况，系统会返回对应的缺失项说明，供用户进一步确认。	校验结果与缺失原因
状态查询	当用户输入有关于状态类的问题时，系统会先对当前运行状态开展读取工作，并且对相关信息进行摘要化展示	如果状态文件出现缺失，系统就会返回未知状态，或者给出对应的后退信息。	运行状态摘要
结果解释	系统会对结果字段开展解析工作，并且结合阈值规则，对严重程度判定情况进行说明。	结果字段不完整时提示依据不足	严重度、触发测量值和建议
知识库重建	在资料完成上传并且触发重建流程之后，系统会返回对应的构建统计信息。	如果资料格式并不受支持，或者构建过程当中出现失败情况，系统会返回相应的错误说明，供用户进一步确认。	对文档数量、知识分块数量以及知识库构建状态开展统一汇总与展示工作

3.5.1 需求边界与验收依据

为了避免系统需求只停留在概念性说明层面，本文在需求分析阶段进一步明确了各类

功能所对应的验收依据。参数问答类需求的验收重点，并不只是看系统是否可以生成一段回答，而是需要核对回答是否能够对应到知识库来源，是否覆盖用户问题当中的关键术语，以及能否避免脱离资料范围去进行自由扩展。操作预览类需求的验收重点，也不只是看系统是否可以识别用户的操作意图，而是要进一步检查系统是否能够抽取扫描区域、分辨率、检测模式以及离线目录等关键字段，并且在字段缺失时给出明确的澄清提示，而不是直接推动执行流程。状态查询类需求的验收重点，则主要在于系统是否能够读取或回退到可解释的运行状态，并把状态、进度、阶段以及最近结果组织成操作人员能够理解的摘要内容。

检测结果解释类需求同样需要开展单独的约束工作。DAC-3D 主系统输出的结果一般会包括缺陷类型、测量值、检测区域、合格性以及过程文件等信息；如果把这些信息直接展示给用户，那么不仅会提高用户的阅读门槛，也不利于用户判断后续应当采取的操作。因此，助手需要先把结果整理成统一字段，再结合阈值规则以及缺陷说明来生成解释内容。该需求的验收依据应当包括字段是否完整、阈值说明是否保持前后一致、结论是否具有明确依据，以及在依据不足时是否会相应降低判断的确定性。操作指导类需求还需要进一步核查建议是否来自知识资料、技能说明或现场规则，同时也不能允许记忆、检索文档或工具输出自行覆盖系统安全政策。

从软件工程这个角度来看，上述各类需求并不是相互割裂的功能条目。参数问答可以为用户理解系统参数以及流程提供前置支持；操作预览会把自然语言输入转化为可供核查的中间结果；状态查询使用户能够判断 DAC-3D 主系统是否已经具备继续执行的条件；结果解释则帮助用户理解检测输出所对应的具体业务含义。只有当这些需求在同一条交互链路当中形成连续配合时，系统才可以真正体现出“智能交互助手”的应用价值。因此，本文后续的总体设计以及实现章节，都会围绕上述验收依据来开展说明，而不是停留在对功能名称进行简单罗列的层面。

相关内容归纳见表 3-6。

表 3-6 功能需求与验收依据对应表

需求类型	验收关注点	不满足要求的表现	论文验证位置
参数问答	回答内容应当以资料来源作为依据来生成，并且能够对专业参数的具体含义进行解释	只给出泛化回答，无法说明依据	第 5 章检索问答实现，第 6 章功能测试
操作预览	是否能够对关键字段开展抽取工作，并且在相关字段出现缺失时提示对应的缺失项	把自然语言请求直接当作执行结果来进行处理	第 5 章命令生成实现，第 6 章边界测试
状态查询	是否可以对状态、阶段、进度以及最近结果等信息开展统一展示工作	仅对底层字段进行返回，或者无法对状态来源作出相应的说明	第 4 章状态桥接，第 6 章接口测试

续表 3-6 功能需求与验收依据对应表

需求类型	验收关注点	不满足要求的表现	论文验证位置
结果解释	是否可以围绕缺陷类型、测量值以及阈值依据，作出相应的说明	脱离结构化结果生成判断	第 5 章结果解析，第 6 章场景验证
操作指导	是否可以吧知识资料以及现场约束作为依据，对建议内容作出相应说明	给出与设备流程无关的通用建议	第 5 章指导链路，第 6 章功能测试
知识库维护	是否能展示资料统计和重建状态	在资料更新工作完成之后，相关更新内容仍然无法及时同步到回答依据当中并得到体现	第 4 章接口设计，第 6 章接口测试

3.5.2 用例覆盖与测试追踪

从可追踪性这一角度来看，需求分析阶段所提出的各类用例，还需要进一步映射到设计模块、实现模块以及测试项目当中。本文把参数问答对应到知识库构建、混合检索以及模型适配模块；把扫描命令预览对应到意图识别、字段校验以及结构化命令模块；把状态查询对应到运行时适配以及状态桥接模块；把结果解释对应到结果解析以及阈值说明模块；把知识库重建对应到资料上传、文档处理以及索引持久化流程。依靠这种对应关系的建立，第 3 章中提出的需求就不再只是停留在文字描述层面，而是可以在后续章节当中形成相对清楚的实现路径以及验证依据。

测试设计同样需要把上述用例作为主线来开展相关工作。如果测试工作只是验证页面能否正常打开，那就仍然不能说明系统已经真实覆盖 DAC-3D 的应用场景；只有将每个用例进一步转化为输入条件、预期输出、异常路径以及验证位置，才可以据此判断系统是否形成了面向检测流程的智能交互闭环。因此，第 6 章设置了测试用例汇总表，对本节用例进行统一编号以及覆盖情况追踪。

3.6 本章小结

本章主要围绕 DAC-3D 检测业务流程、用户角色、功能需求、非功能需求及典型用例等内容，对系统需求开展了相对系统的分析工作。本文系统并不是一般意义上的通用聊天窗口，而是把参数理解、操作预览、状态查询、结果解释以及知识库维护等具体任务当作对象进行构建的受约束智能交互助手。上述需求同时也为后续总体架构设计、数据结构定义、接口设计以及测试验证工作提供了较为直接的依据。

第 4 章 系统总体设计

在前文已经完成需求分析以及相关技术梳理工作的基础上，本文进一步围绕 DAC-3D 智能交互助手原型的系统结构、功能流程、数据组织以及接口形式，对总体设计工作进行说明和展开。在总体设计阶段当中，本文把系统划分为若干个相对独立、同时又可以相互协同运行的功能模块，这样一来，知识问答、操作预览、结果解释以及状态查询等功能便可以在统一框架下得以实现。为了能够更加清楚地呈现系统的总体架构，各模块之间的对应关系以及协作关系如图 4-1 所示。

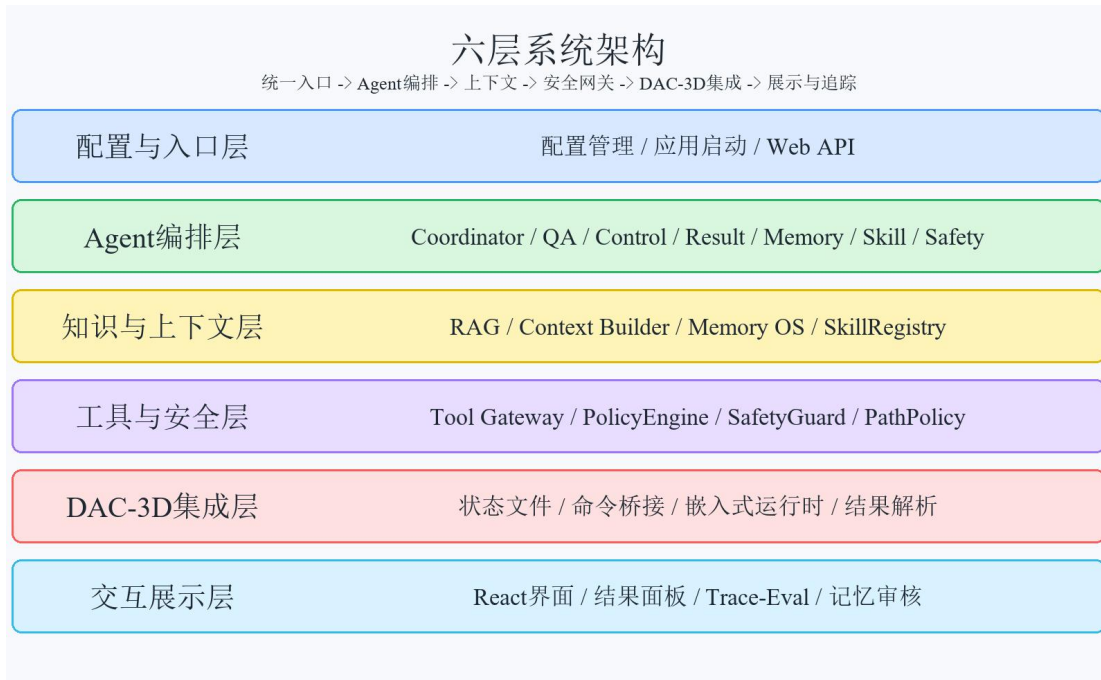


图 4-1 系统总体架构图

相关内容归纳见表 4-1。

表 4-1 系统核心模块职责分工

模块	输入	输出	主要职责
配置与入口层	环境变量、启动参数、运行目录	应用实例和统一接口	完成对配置读取、依赖装配以及运行模式切换等环节的处理工作
Agent 编排层	用户消息、会话状态	任务分派、工具计划和响应负载	对问答、控制、结果、记忆、技能以及安全链路开展统一协调工作
知识与上下文层	文档、技能、记忆、运行状态	检索片段和 ContextBundle	对当前任务所需要的上下文内容开展选择、压缩以及隔离处理工作
工具与安全层	命令预览和工具参数	风险决策结果、验证结果以及命令生命周期信息	执行 Tool Gateway、PolicyEngine 和 SafetyGuard 约束
DAC-3D 集成层	运行状态文件、受控命令	状态摘要、命令回执和规范化检测结果	隔离运行适配和结果解析，支持模拟环境、状态文件桥接以及受控命令提交

续表 4-1 系统核心模块职责分工

模块	输入	输出	主要职责
交互展示层	Agent 响应、来源依据、命令预览和运行状态	网页接口、前端视图和用户确认结果	通过 FastAPI 和 React 展示多轮问答、来源依据、结构化命令、安全审批以及 Trace/Eval 信息

4.1 系统设计目标与设计原则

为了能够使系统既满足 DAC-3D 场景下的实际使用需求，又可以对后续功能扩展形成支撑，本文在总体设计过程当中，主要遵循知识受约束、Agent 分层编排、工具调用受控、上下文可信隔离以及演进兼容这五项原则。

知识受约束原则，主要指系统当中的参数问答、操作指导以及结果解释等功能，应当尽量把检索获得的领域资料或结构化结果字段当作基础来生成回答，而不是完全依靠模型开展自由生成。DAC-3D 属于专业检测软件，当用户询问参数含义、故障处理方法或者结果判断依据时，如果系统输出的内容脱离了手册说明、规则约束或者检测结果本身，就容易使操作人员产生理解上的偏差。因此，本文在设计中要求问答链路尽可能先进行知识库检索，结果解释链路也尽可能先进行结构化结果解析，使回答内容可以与资料来源、字段信息以及规则依据保持一致。

Agent 分层编排原则主要强调的是，不同类别的用户请求需要进入与之对应的处理链路。系统不再把所有输入直接交给单一规则路由或单一提示词去处理，而是借助统一入口先对任务类型进行判断，然后分别交由问答、控制、结果、记忆、技能以及安全相关能力继续开展处理。依靠这种分层方式，可以把只读问答请求与高风险控制请求进行有效区分，避免工业场景中的操作请求被普通聊天逻辑错误处理。

工具调用受控原则主要强调的是，凡是可能影响 DAC-3D 运行状态的操作，都不能让 LLM 直接写入命令文件，也不能让它直接调用主系统能力，而是需要统一经过 Tool Gateway、PolicyEngine、SafetyGuard、schema 校验、路径白名单、风险分类、确认 token 以及命令生命周期记录等环节。这样一来，即使模型输出、检索资料、长期记忆或者技能说明当中包含错误指令，也难以越过代码层面所设置的安全边界。

上下文可信隔离以及演进兼容原则，主要服务于 Agent 系统后续的长期维护工作。系统会依据当前任务，对运行状态、检索资料、技能说明以及记忆命中进行筛选和压缩，而不是把所有文档、历史记录以及工具说明一次性放入提示词当中。同时，每一段上下文都应当明确标注来源以及信任等级：内置安全策略用于约束执行过程，工具输出只能被当作观察结果，检索文档以及记忆只能提供依据或背景，不能被赋予工具权限。保留模拟运行环境、状态文件桥接、命令桥接、记忆审核以及本地评测入口，都是为了能够支撑系统由原型阶段逐步过渡到更加完整的联机形态。

4.2 系统总体架构设计

DAC-3D 智能交互助手可以被划分为配置与入口层、Agent 编排层、知识与上下文层、工具与安全层、DAC-3D 集成层以及交互展示层这六个层次。开展这样的层次划分，并不是为了额外提高系统复杂度，而是为了能够让“理解用户问题”“选择依据”“调用工具”“保证安全”“接入主系统”以及“展示结果”等环节，都拥有相对清楚的职责边界^{[24][25]}。

配置与入口层主要由配置管理模块以及助手应用入口模块共同组成。配置管理模块负责对模型服务、向量库路径、知识文档路径、前端目录、临时目录、Agent 模型参数、记忆开关、安全开关以及运行时桥接路径等配置内容开展统一管理工作，从而保证运行参数可以拥有集中的维护入口；助手应用入口模块则负责完成系统启动、依赖装配、请求接收以及不同运行模式之间的切换处理工作。

Agent 编排层主要由统一 Agent 入口、会话状态以及专家能力控制器共同构成。该层负责对用户请求所对应的任务属性开展判断，明确它属于问答、状态、结果、控制、记忆、技能还是安全审查任务，并且据此选择后续处理链路。它的作用并不是替代安全模块，而是把模型的语义理解能力限定在受控的任务分派框架当中。

知识与上下文层主要由知识库构建、混合检索、Context Builder、SkillRegistry 以及 Memory OS 等部分共同组成。其中，知识库负责提供可以追溯的文档依据；Context Builder 会对本轮任务所需要的状态、技能、记忆以及安全策略开展选择与压缩；SkillRegistry 则按需提供流程化技能说明；Memory OS 负责保存会话历史，并且生成需要审核的记忆补丁。该层重点解决的问题，是进一步明确“模型在本轮回答中应当依据什么来回答”。

工具与安全层主要由 Tool Gateway、PolicyEngine、SafetyGuard、PathPolicy 以及命令生命周期状态机共同构成。该层重点用于解决“模型所计划调用的工具是否允许执行”这一问题。其中，读取状态以及读取结果被归为只读操作，命令预览被归为中风险操作；启动在线扫描、离线检测或停止检测等动作则被归为高风险操作，必须在完成相关校验以及确认之后，才可以继续提交。

DAC-3D 集成层主要由运行时适配模块以及结果解析模块共同组成。前者负责同 DAC-3D 运行时开展交互，目前既可以支持模拟运行环境，也可以基于状态文件桥接来读取主系统状态，同时还支持在安全网关完成批准后，凭借命令桥接或嵌入式运行时提交受控命令；后者则负责把检测结果映射成统一字段，并且在必要情况下给出对应的判定依据。

交互展示层主要由网页接口模块、frontend/以及必要的兼容调试入口共同构成。其中，网页接口模块负责向外提供聊天、状态、知识库管理、命令审批、评测运行、Trace 读取、目标管理以及记忆补丁审核等接口，frontend/则负责对主交互界面开展实现工作。该层主要面向系统结果的可视化呈现，重点处理用户如何查看并且理解结果这一问题，具体包括多轮消息展示、来源依据展示、结构化命令预览、运行时信息展示、安全审批以及评测信息展示等内容。

总体来看，这六层结构分别回应了系统如何启动、如何开展任务分派、如何选择依据、如何约束工具调用、如何接入 DAC-3D 以及如何展示结果等问题。依靠这种分层设计，系统不再只是附加在界面之上的聊天窗口，而是进一步形成了由知识、上下文、安全、工具以及审计共同支撑的任务型软件原型。

4.3 系统功能流程设计

系统功能流程把用户自然语言输入当作起点，并且以统一响应展示作为终点。它不同于直接把问题发送给模型、再返回单句回答的方式，本文系统会先进入 Agent 运行时，然后依据任务类型选择相应的上下文、技能、记忆以及工具链路。依靠这一流程，参数问答、状态查询、结果解释以及执行类操作，都可以在适当的约束条件下分别完成。

用户在前端完成消息输入后，请求会先进入应用入口层。系统结合当前会话信息，对请求开展基础整理工作，并将其交由 Agent Runtime 判断具体任务类型。随后，Context Builder 会根据本轮任务需要，选择运行状态、安全策略、技能说明以及记忆命中内容，再把这些信息压缩成本轮提示上下文。

当请求被判定为状态查询时，系统会借助只读工具对 DAC-3D 的运行状态进行读取；如果请求属于参数问答或操作指导，系统就会进入知识检索以及技能辅助链路；如果请求属于操作类任务，系统会先生成命令预览，再依次交由 Tool Gateway、PolicyEngine、SafetyGuard 以及 PathPolicy 开展校验工作，并且在必要情况下等待用户确认；如果请求属于结果解释，系统会先获取检测结果，并对相关字段进行规范化处理，再结合缺陷规则来生成说明内容。

由此来看，系统的整体流程可以被概括为“任务分派—上下文选择—受控工具调用—结果生成—审计记录”。这一流程的重点并不是环节数量本身，而是要依照任务风险等级划定相应的处理边界：问答任务需要强调依据可追溯，状态任务需要强调只读访问，结果解释需要强调字段映射，控制任务则需要强调确认机制以及审计记录。

在功能流程设计过程当中，异常处理以及回退逻辑同样需要提前被纳入设计范围。例如，如果知识库处在不可用状态，问答链路应当返回清楚且便于理解的提示信息；如果用户给出的参数并不完整，命令链路需要据此生成对应的澄清问题；如果命令涉及高风险执行，系统需要创建 pending preview，并且等待用户确认；如果记忆或检索资料当中出现试图绕过安全策略的内容，系统需要把它视为不可信上下文，并由策略层开展拦截；如果 DAC-3D 主系统状态文件不可用，运行时查询也可以回退到模拟运行环境或样例数据。上述处理虽然并不属于主流程本身，但对于保障原型系统能够稳定运行，仍然具有较为重要的作用。图 4-2 进一步说明了系统请求识别以及任务路由之间的对应关系。

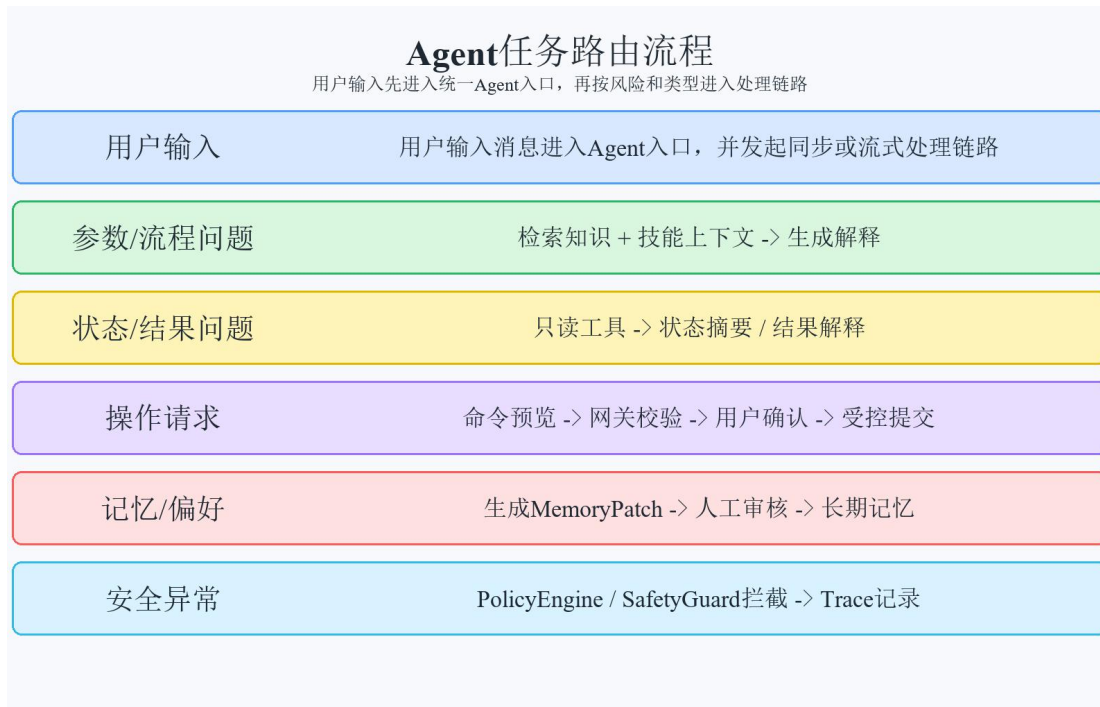


图 4-2 任务路由流程图

4.4 核心数据结构设计

为了保证各模块之间的输入以及输出关系更加清楚，系统对若干核心数据结构进行了定义工作，用来承载检索结果、Agent 响应、上下文包、技能说明、记忆补丁、工具描述、风险决策、结构化命令、运行时状态以及检测结果等信息。依靠这些统一的数据结构，各模块可以凭借稳定字段开展协作，而不必直接耦合到彼此的内部实现当中。

检索结果结构主要用于记录片段文本、资料来源、章节信息、文档类型、片段编号以及相似度；Agent 响应结构则用于记录回答文本、意图、结构化负载、命令预览、状态摘要以及工具调用信息；上下文工程结构会记录本轮所选用的运行状态、安全策略、技能说明、记忆命中、信任等级以及被排除的上下文。依靠这些结构，系统可以对每轮回答所依据的内容以及上下文来源开展追踪。

命令预览以及工具网关结构，是操作链路当中的核心组成部分。命令预览结构主要负责对动作类型、扫描区域、分辨率、检测模式、缺失字段以及风险提示进行保存；ToolDescriptor、ToolInvocationResult 以及 RiskDecision 等结构，则用于描述工具元数据、工具调用结果、风险等级以及确认要求。记忆以及评测结构用于保存 MemoryPatch、Trace 以及 EvalCase，这样可以使长期记忆写入以及评测用例都具备审核和追踪依据。

这些数据结构的设计，会对系统实现工作形成较为直接的支撑作用。如果没有统一的数据结构，模块之间通常只能借助临时字典或自由文本来进行信息传递，这样不仅会让模块边界逐步变得模糊，还会进一步增加测试以及调试工作的复杂度。在选用统一结构之后，模块之间的交互形式可以保持更加稳定，前端可以依据字段类型有选择地展示相关内容，

安全模块也可以依据字段开展确定性校验，而不需要在每一次调用时重新判断返回结果的格式。

在上述核心结构之外，系统仍然保留了来源详情、知识库摘要、命令历史、目标条目以及评测草稿等辅助数据，用来补充核心链路之外的展示与追踪信息。其中，来源详情主要用于记录来源文件、标题、章节、文档类型、检索分数以及片段编号；命令历史则用于记录 `preview_created`、`validation_passed`、`awaiting_confirmation`、`confirmed` 以及 `submitted` 等生命周期状态；评测草稿用于把近期 `Trace` 整理为待审核的回归用例。借助这些结构，前端或客户端在展示回答内容时，不仅可以呈现文本结果，还可以同步展示对应的依据来源、构建状态、安全审批状态以及评测追踪信息。

从测试工作这个角度来看，较为稳定的数据结构同样可以为自动化验证工作提供直接的支撑。测试脚本可以围绕字段是否存在、类型是否正确、风险等级是否准确、确认要求是否已经触发以及安全拦截是否生效等条件进行断言，而不必完全依靠整段自然语言回答。对于软件类毕业设计而言，这样的设计不仅体现了程序在可维护性方面的特性，同时也进一步说明了系统具备较好的可验证性。表 4-2 对主要数据结构的字段以及它们的作用进行了归纳。

为了能够更加清楚地说明数据结构设计图，各类结构之间所形成的关联关系如图 4-3 所示。

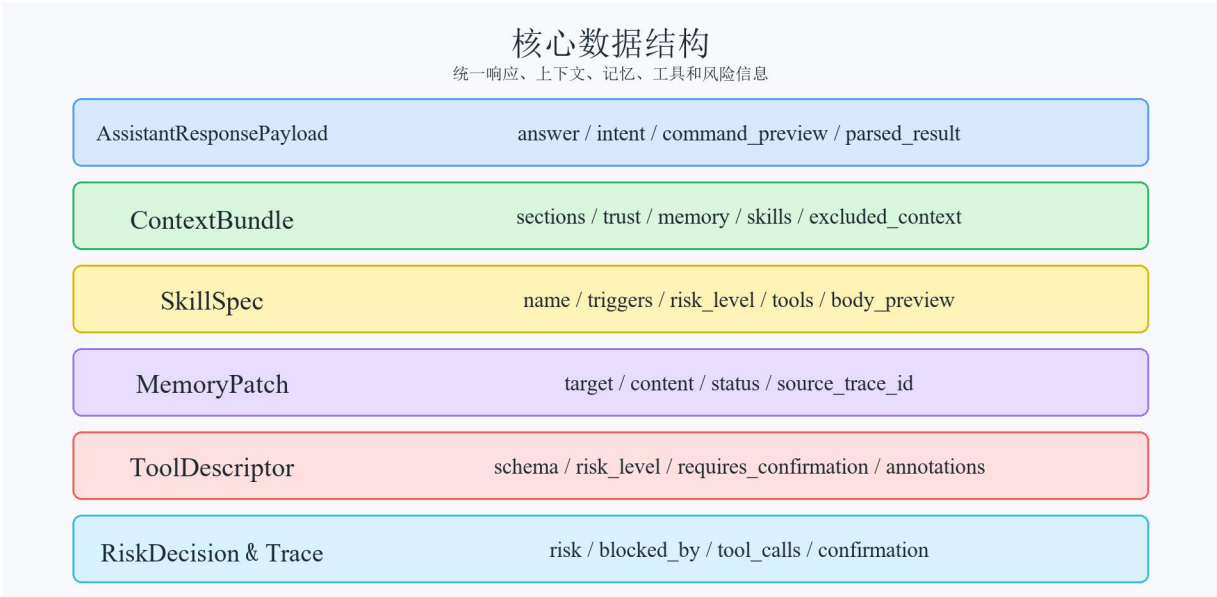


图 4-3 数据结构设计图

相关内容归纳见表 4-2。

表 4-2 核心数据结构说明

数据结构	主要字段	作用	使用位置
RetrievalItem	片段文本、资料来源、章节和相似度	承载检索片段与来源	问答、指导、解释

续表 4-2 核心数据结构说明

数据结构	主要字段	作用	使用位置
ContextBundle	sections、trust、memory、skills 和 limits	记录本轮上下文工程结果	Agent 提示构建与调试
SkillSpec	名称、触发词、风险等级、相关工具以及正文摘要	描述本地技能目录	技能发现与按需加载
MemoryPatch	目标、内容、原因、状态和来源 Trace	保存待审核长期记忆写入	记忆补丁审核
ToolDescriptor	工具名、schema、风险等级和 annotations	声明可用受控工具	Tool Gateway 与前端提示

4.5 接口设计

当前系统中的接口大体可以划分为五个类别，分别对应聊天接口、运行时接口、知识库管理接口、Agent 工作台接口以及 DAC-3D 受控工具接口。聊天接口主要承担以同步方式或流式方式返回响应的工作；运行时接口用于展示模型、向量库、文档以及 DAC-3D 状态；知识库管理接口用于支持摘要查看以及重建；Agent 工作台接口用于承载命令审批、Trace 读取、评测运行、目标管理以及记忆补丁审核等工作；受控工具接口则用于支撑状态读取、结果读取、命令预览、命令验证以及命令提交。

这样的接口划分使前端不需要直接访问后端内部模块，从而得以实现展示层与执行逻辑之间的解耦。同步聊天接口较为适宜调试以及自动化断言，流式接口适用于真实交互场景，Agent 工作台接口则使系统拥有命令确认、记忆审核以及评测追踪等能力。这样一来，前端所展示的内容不再只是文本回答，还会包括来源、状态、命令风险以及审计信息。

接口设计同样需要提前对异常返回方式进行约束，以便错误状态可以得到稳定处理。在参数缺失、知识库不可用、模型调用失败、设备状态异常、路径越权、确认 token 过期以及安全策略拦截等场景当中，接口不适宜只返回程序层面的错误信息，而应尽量生成前端能够识别的错误说明或回退结果，使前端可以把异常状态转换为用户能够理解的提示信息，而不是把底层报错直接暴露给使用者。接口层借助统一的请求结构以及响应结构，把前端、Agent 运行时、核心服务以及运行时适配模块连接起来，同时为后续前后端消息流转图当中的实现链路提供相应的设计依据。

在聊天接口以及运行时接口之外，系统还继续设置了健康检查接口、命令审批接口、评测接口、Trace 接口以及记忆补丁审核接口。鉴于这些接口都以统一的 Web 形式向外暴露，当前原型既可以在不依赖 DAC-3D 系统的情况下进行单独启动以及单独测试，也可以由 DAC-3D 系统依靠网页跳转方式来完成调用。这样一来，助手模块就形成了相对清楚的独立功能边界，同时还为前后端联调、客户端调用以及 Agent 回归验证提供了一套一致的接口基础。

4.6 部署与集成设计

4.6.1 本地独立运行模式

本地独立运行模式主要面向开发、演示以及自动化测试等场景。在这一模式当中，`dac3d_iim_assistant`可以在不依赖 DAC-3D 主系统的情况下独立完成启动，它的运行时数据则由模拟运行环境、样例知识库、状态文件或本地 Agent 验证模型来提供。借助这一设计，可以在开发阶段降低对真实设备、相机、运动控制以及数据库环境的依赖程度，使知识库问答、Agent 分派、命令预览、接口返回、安全审批、记忆补丁以及前端展示等功能先行形成较为稳定的闭环。

4.6.2 DAC-3D 主系统跳转网页助手模式

在与 DAC-3D 主系统开展集成时，主界面以及状态同步模块当中已经预先保留了 AI 按钮以及网页入口，主系统可以依靠该入口打开本地网页助手，并且由网页助手借助 FastAPI 接口去调用后端 Agent 能力。这个模式的主要优势在于它的侵入性相对较低，主系统不需要直接承载模型调用、向量检索、Agent 编排以及 React 前端展示等逻辑，因此可以在不影响原有检测界面的前提下，对智能助手能力开展相关验证工作。

4.6.3 状态文件桥接模式

状态文件桥接模式主要用于在不额外开放设备通信协议的前提下，来实现运行状态层面的同步工作。DAC-3D 主系统会在启动、检测初始化、离线检测、样品结果生成、停止以及完成等环节当中调用状态写入流程，并且写出 JSON 状态文件；助手后端在接收到状态查询或结果解释请求之后，会借助受控只读工具去读取该文件，再把它映射为运行状态摘要或 `InspectionResult`。如果文件不存在、字段出现缺失，或者解析过程发生失败，那么系统就需要返回较为明确的未知状态、`error` 或回退状态。

相关内容归纳见表 4-3。

表 4-3 状态文件字段说明表

字段	含义	缺失处理	使用位置
状态	运行状态主要用来表示任务当前所处的阶段，当中包括空闲、运行中、已完成以及异常等状态	默认为未知状态	状态面板、状态问答
进度	检测进度百分比	默认为 0	前端进度展示
阶段	主要用于说明主系统当前所处的运行阶段，比如检测初始化、样品结果更新以及检测完成等	为空时只显示状态和消息	状态摘要、调试定位
盘编号	当前盘编号	为空时显示未绑定盘编号	检测上下文展示

续表 4-3 状态文件字段说明表

字段	含义	缺失处理	使用位置
离线模式、离线原图目录	是否离线模式及离线原图目录	为空时按在线或未知模式处理	离线检测说明
最近结果字段	主要用于表示最近一个样品所对应的检测结果，其中包括质量、缺陷数量以及解析结果等字段信息	为空时提示尚无结果	结果解释入口
结果历史字段	最近检测结果列表主要用于保存近期产生的检测结果，最多可以保留 144 个样品对应的相关记录	为空时只使用最近结果字段	结果追踪和后续扩展
source 以及 updated_at 字段	状态来源和更新时间	记录当前回退来源	调试和测试

4.6.4 受控命令桥接模式

除状态读取之外，项目还设置了命令桥接以及嵌入式运行时桥接这两类受控提交方式。需要说明的是，助手并不会允许 LLM 直接对命令文件进行写入。操作请求必须先生成结构化的命令预览，并且在当前会话中进入 pending lifecycle；随后再由 Tool Gateway 完成工具注册、schema、路径、风险以及确认 token 等方面的校验工作。只有在 preview_id 以及 confirmation_token 都通过校验之后，submit_command 才会把命令提交到命令文件桥接、embedded bridge 或 mock runtime。

相关内容归纳见表 4-4。

表 4-4 命令桥接动作说明表

动作	来源字段	主系统处理方式	安全约束
在线扫描动作	扫描区域、分辨率、模式和确认 token	只有在经过 Tool Gateway 批准之后，系统才会对启动检测流程或在线扫描动作进行调用	当任务已经处于运行中状态时，系统会拒绝重复启动；相关操作也必须经过显式确认。
离线检测动作	图像目录、检测模式和确认 token	在路径校验已经通过之后，系统会把它提交给离线检测桥接流程	目录必须处在白名单所允许的范围内；如果目录为空，或者内容存在不完整情况，那么系统应当拒绝执行。
停止检测动作	停止意图和确认 token	经确认后转交主系统停止流程	高风险控制动作需要先完成确认，并且记录对应的 Trace。

续表 4-4 命令桥接动作说明表

动作	来源字段	主系统处理方式	安全约束
状态查询动作	状态查询意图	读取状态文件或嵌入式状态快照	只读工具，不允许触发控制命令。
最近结果读取	结果查询意图或样品编号	读取最近检测结果并归一化字段	仅对只读工具进行调用；如果返回结果仍然不足，就提示当前依据不足。

4.6.5 模拟运行环境回退模式

模拟运行环境回退模式主要用来应对真实设备、状态文件、外部模型服务以及 Agent 依赖暂时不可用的情况，从而继续维持系统基本的演示与测试能力。该模式并不能替代真实联机测试，但可以保证系统在开发阶段依然能够围绕消息路由、接口返回、结构化命令、安全拦截、Trace 记录以及前端展示等内容开展相应的验证工作。

相关内容归纳见表 4-5。

表 4-5 异常处理与回退策略表

异常场景	检测方式	回退策略	用户可见结果
知识库未构建	知识库摘要内容为空，或者与之对应的知识库清单文件并不存在	返回资料缺失提示	提示先上传或构建知识库
操作参数缺失	缺失字段非空	不生成可以提交的命令，只生成用于补全相关信息的澄清提示	提示补充扫描区域、分辨率或模式
路径越权	PathPolicy 校验失败	阻断命令验证或提交	提示目录不在允许范围内
确认失效	confirmation_token 过期、不匹配或重复消费	拒绝 submit_command	提示重新生成命令预览并确认
Prompt Injection	当检索资料、用户输入或工具输出当中出现试图绕过确认流程的文本时	标记注入信号并由策略层拦截	提示请求违反安全策略

4.6.6 图表化表达与设计约束

在本文的总体设计当中，图表主要被当作界定系统边界以及说明流程关系的工程化工具来使用：系统架构图用于展示前端、Agent 运行时、上下文工程、工具安全层、DAC-3D 适配层以及主检测系统之间如何形成调用关系；任务路由图用于对用户输入进入不同处理

链路时所依赖的判断条件进行说明；数据结构图用于说明响应、上下文、记忆、技能以及工具调用之间所形成的关联；命令生成流程图则用于进一步说明自然语言请求为何需要依次经过预览、校验、确认以及提交等环节。

在开展图表设计工作时，需要对图中所承载的信息密度进行合理控制，使图形可以更好地服务于结构说明，而不是直接替代正文内容。对于流程图来说，图内节点应选用相对简短的中文表述，例如“接收用户请求”“识别任务类型”“检索知识片段”“生成命令预览”“读取运行状态”等，避免把完整段落直接放置到图形当中。判断节点只需要保留必要的条件分支，较为详细的约束规则则应当放在正文部分进行解释。借助这种处理方式，图中文字字号可以控制在略小于正文的范围内，同时也能够避免图像在缩放之后出现阅读困难的问题。对于表格，本文优先运用表格来承载需求、接口、数据结构以及测试项之间的对应关系，表内文字应尽量保持为短句，较长的解释内容则放在表前或表后的正文中展开说明。

这种图表化的表达方式，同样可以帮助更加准确地呈现软件设计以及实现过程当中的实际投入。就本文系统来看，相关工作量并不只是体现为若干程序文件的编写，而是需要围绕需求分析、模块划分、数据结构设计、运行时适配、前端展示以及测试验证等方面，完成一套连续的工程链路。把这些内容进一步整理成图表之后，评阅者可以更加直观地理解系统结构、处理流程以及验证路径，同时也能够避免论文只是被呈现为程序文本清单。

相关内容归纳见表 4-6。

表 4-6 图表化表达约束说明表

表达对象	推荐表达方式	图表文字控制	正文说明重点
系统边界	总体架构图	节点用中文模块名	说明模块职责和调用关系
请求处理	任务路由流程图	节点用动作短语	说明不同任务的进入条件
状态同步	时序或流程图	保留关键状态节点	说明状态写出、读取和回退
命令生成	流程图加字段表	判断条件简写	说明安全校验和缺失项提示
测试验证	测试表和场景表	表内用短句	说明覆盖范围和扩展测试项

4.7 本章小结

本章以前文的需求分析以及技术路线为基础，对 DAC-3D 智能交互助手原型的总体架构、功能流程、数据结构以及接口设计进行了较为系统的说明。借助这些内容可以看出，本文并没有把系统设计成简单的通用聊天程序，而是围绕工业任务型助手的实现逻辑，把知识检索、任务处理、DAC-3D 适配以及前端交互组织成可以相互协作的多层结构。依靠这一设计，系统既能够满足当前原型演示以及程序验证方面的要求，也为后续开展真实设备联调、知识库扩展以及界面嵌入等工作保留了相对清楚的工程边界。

此外，本章中所给出的分层结构、流程组织以及接口形式，也为第 5 章开展各个模块

的具体实现工作提供了较为直接的依据。后续章节将不再继续从整体层面对系统应当如何进行设计展开讨论，而是转向知识库、检索增强问答、意图识别以及命令生成、结果解析、交互界面等具体模块，进一步说明各部分如何在既定架构之下得以实现，以及这些模块又如何共同支撑整个原型系统的运行。

第 5 章 系统实现与运行效果

5.1 后端服务与应用入口实现

5.1.1 配置管理与应用启动

助手后端选用“配置管理—Agent 装配—请求分发”的启动方式。系统在启动阶段会先对基础目录、知识库路径、模型服务、前端资源、运行时桥接路径、记忆目录、Trace 路径以及命令确认有效期等配置项进行读取，并且在这些配置的基础上完成检索器、模型客户端、Agent Runtime、Context Builder、SkillRegistry、Memory OS、Tool Gateway 以及 DAC-3D 适配层的装配工作，最后再由接口层统一向外提供聊天、状态查询、知识库管理、命令审批、记忆审核以及评测运行等能力。

5.1.2 FastAPI 路由与服务组织

Web API 层主要负责把后端能力组织成较为稳定的对外接口，具体包括健康检查、运行时信息、知识库摘要、同步聊天、流式聊天、知识库重建、命令审批、Trace 读取、评测运行、目标管理以及记忆补丁审核等内容。接口层本身并不直接承担检索、结果解释或安全判断等逻辑，而是会把请求转交给应用编排层以及 Agent 运行时去处理，随后再将统一响应返回给前端。

5.1.3 模型服务与运行时装配

在模型以及运行时装配方面，系统保留了本地回退、外部大语言模型服务以及本地 Agent 验证模型这三类能力。其中，外部模型主要承担真实生成工作，本地回退用于支持无网络条件下的演示，本地 Agent 验证模型则用于在不依赖外部模型的情况下，对工具调用路径开展相关的验证工作。运行时方面，系统继续保留模拟环境、状态文件桥接、命令文件桥接、嵌入式桥接、命令生命周期以及 Trace/Eval 入口，使原型可以逐步过渡到联机运行形态。

5.1.4 主系统流程与助手接入

DAC-3D 主检测系统主要由主界面、调试控制以及图像处理等部分共同组成。其中，主界面承担运行按钮、离线检测、状态写入以及命令轮询等方面的管理工作；图像处理流程则负责对离线原图读取、三相机图像保存、配准融合、已有缺陷检测、结果文件输出以及样品结果回传等环节开展统一处理。上述检测模型以及图像处理流程属于 DAC-3D 主系统原本已经具备的能力，本文不将这些内容纳入课题实现范围；嵌入式助手面板以及命令桥接仅作为受控入口，在 Tool Gateway 完成确认并通过之后，用于衔接主系统已有能力。

相关内容归纳见表 5-1。

表 5-1 关键模块职责与流程对应表

模块	模块职责	论文中对应内容
主系统启动模块	对主界面、调试控制以及图像处理这三个进程开展启动工作	DAC-3D 主系统 运行流程
主界面与状态同步 模块	对 144 点位扫描、离线检测、状态写入以及命令轮询等流程开展统一组织与衔接工作	主系统集成与状态 同步
统一 Agent 入口	接收用户消息，并且把它分派到问答、控制、结果、记忆、技能以及安全等链路当中	Agent 运行时实 现
Context Builder	对运行状态、技能、记忆以及安全策略开展选择和压缩处理工作	上下文工程实现
Tool Gateway 与 SafetyGuard	对工具注册、schema、路径、风险、确认以及生命周期等内容开展校验工作	安全执行与命令 审批实现
Trace/Eval 模块	对运行轨迹开展记录工作，并且执行本地回归测试以及安全评测工作	测试与评估闭环
混合检索模块	对语义检索、词项统计、融合排序以及来源组织等环节开展统一组织与处理工作，从而保证检索结果与来源依据可以协调呈现	检索增强问答实 现
意图识别模块	对问答、操作、状态、解释以及指导类请求开展识别与处理工作	意图识别实现
结构化命令模块	对字段抽取、缺失项校验以及命令预览生成等环节开展统一组织与处理工作	结构化命令生成 实现
运行时适配模块	对模拟运行、状态文件、命令文件以及嵌入式桥接等内容开展统一的适配处理工作	DAC-3D 适配实 现
结果解析模块	对检测结果字段开展统一整理工作，并且在此基础上生成对应的严重度判定依据	结果解析实现
Web 交互模块	完成接口封装工作，并在此基础上来实现流式输出、来源展示、命令展示以及状态展示等方面的功能	网页前端交互界 面实现

为了能够更加清楚地说明用户请求处理流程当中各个环节之间的关系，相关内容如图 5-1 所示。

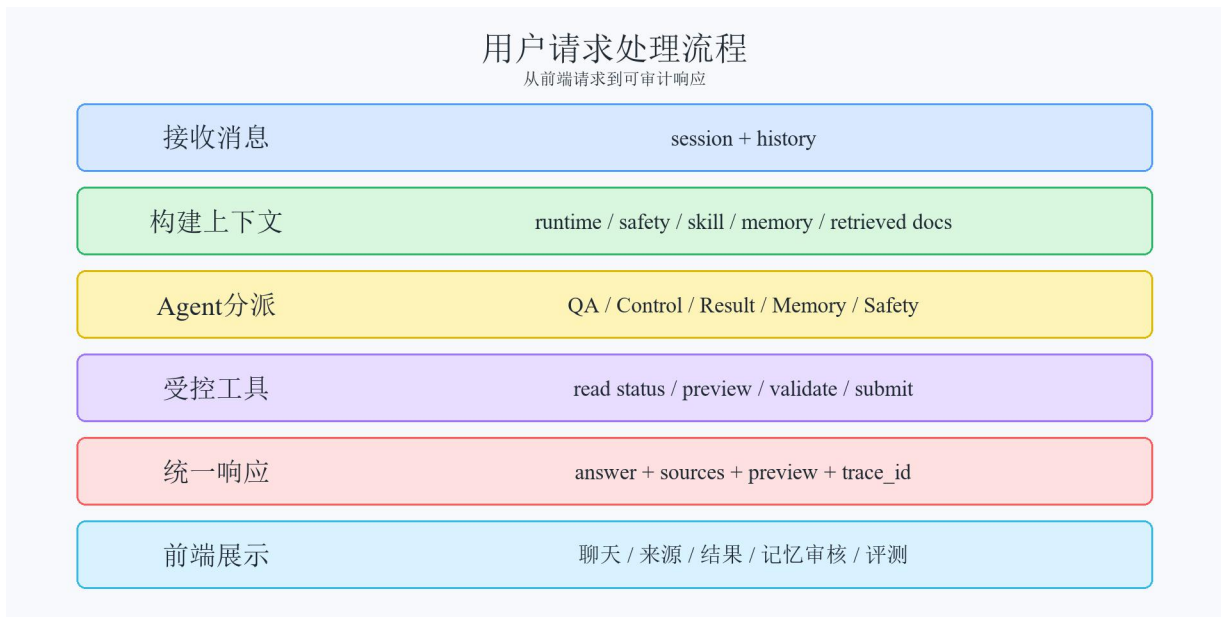


图 5-1 用户请求处理流程图

为了能够更加直观地说明系统前端交互界面的组成要素，以及各个交互环节之间所形成的关系，相关内容如图 5-2 所示。

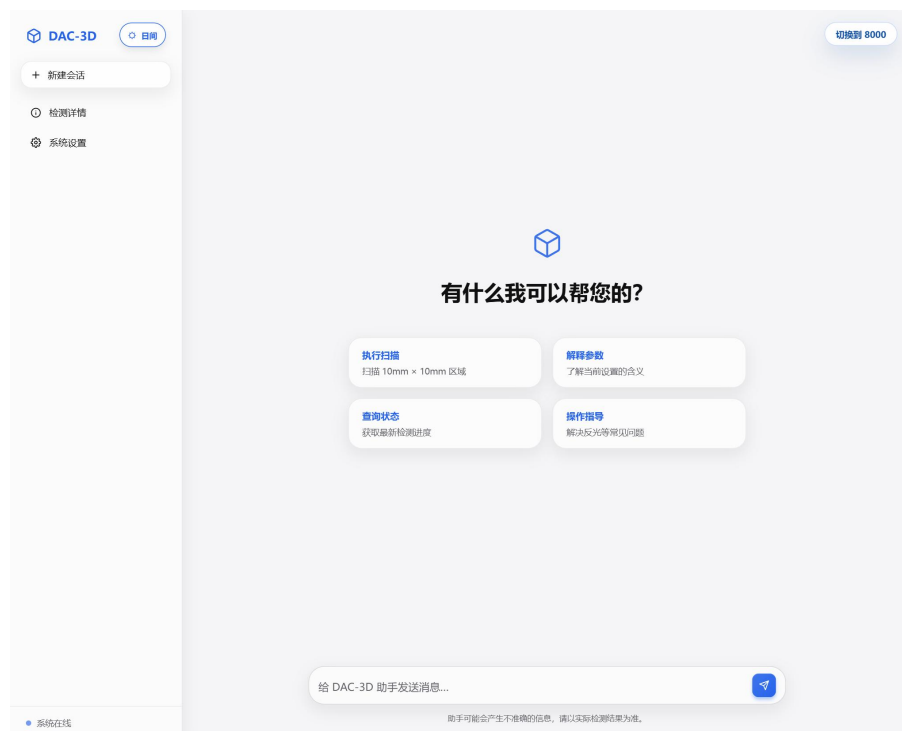


图 5-2 系统前端交互界面示意

相关内容归纳见表 5-2。

表 5-2 核心实现模块完成状态表

文件	所在层次	主要职责	状态
agent_runtime.py	Agent 编排层	对入口、会话装配等环节开展统一组织工作	已完成
context_engineering/	知识与上下文层	选择、压缩并隔离本轮上下文	已完成
memory/	记忆层	对会话历史开展保存工作，并且生成待审核的记忆补丁	已完成
skill_system/	技能层	发现、选择和读取本地技能	已完成
tool_gateway/ 与 safety/	工具与安全层	对命令预览、验证、确认、提交以及安全拦截等环节开展处理工作	已完成

从实现形态来看，系统并不是围绕某一个单一算法来展开设计，而是由多个承担不同职责的模块协同构成。其中，知识库主要用于提供文档依据，Agent 运行时负责统一入口以及任务分派，上下文工程负责选择本轮交互所依赖的相关依据，Tool Gateway 以及安全模块负责对工具调用进行约束，DAC-3D 适配与结果解析模块负责隔离主系统差异，前端模块则负责把文本、来源、命令以及状态组织成用户可以理解的界面结果。

为了能够更加清晰地呈现系统部署结构以及各模块之间所形成的调用关系，相关内容如图 5-3 所示。

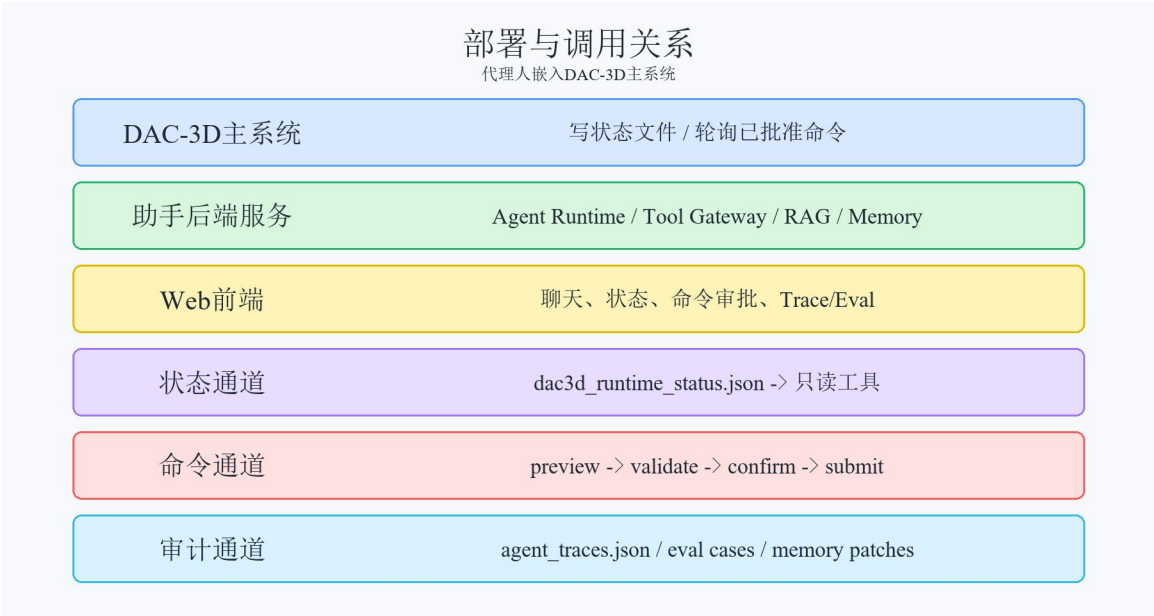


图 5-3 系统部署与调用关系示意图

5.2 知识库构建与检索问答实现

5.2.1 知识库资料整理与构建

知识库模块是本文原型系统当中较为基础的组成部分，同时也是参数问答、操作指导以及结果解释等能力能够成立的重要前提。如果系统缺少稳定的文档知识来源，那么后续问答以及建议生成链路就会更多地依赖模型自身记忆，回答内容的可控性以及可追溯性也会随之下降。鉴于这一考虑，本文首先实现了本地知识库搭建模块，用于对 DAC-3D 相关文档进行统一处理，并且把这些文档转换为可以被检索的知识集合。

在当前实现当中，知识库构建模块主要承担知识库搭建过程中的关键处理工作。该模块可以从知识资料目录中加载 Markdown、纯文本、PDF 以及 DOCX 等多种格式的文档，并且运用“内容探测优先、文件后缀兜底”的方式，对真实文档格式开展识别与归类。选用这一策略的主要缘由在于，实际资料来源往往并不统一：部分文档可能经过人工整理，部分文档则会直接来自原始说明资料，文件命名以及后缀并不总是完全规范。如果只是依赖后缀来判断格式，就容易导致有效内容无法被正确识别，因此本文在该模块当中会优先关注文本内容本身是否具备可识别性。

5.2.2 文档分块与索引持久化

在完成文档加载工作之后，系统会继续对原始文本开展标准化清洗工作，主要包括去除噪声内容、梳理标题层级以及保留基础编号结构等处理。文本清洗完成以后，模块并不会把整篇文档直接当作单一大块来进行向量化，而是会凭借标题、编号结构以及句子边界，对文本进行分块处理。之所以选用这种处理方式，主要是为了能够在检索粒度以及上下文完整性之间取得相对平衡。如果分块过大，那么检索结果当中就容易混入较多无关信息；如果分块过小，又可能造成上下文不够完整，进而影响后续回答质量。在当前实现当中，每个分块都会保留 `source`、`title`、`section`、`document_type` 等元信息，便于后续在回答中展示来源以及章节位置，同时也便于知识库维护时开展统计与追踪。

生成完成后的向量以及相应元数据，最终会以持久化方式写入本地向量库目录，以便后续检索模块可以开展调用工作^[28]。同时，系统还会对知识库搭建历史、文档数量以及分块数量进行记录，从而为知识库的基本运维管理工作提供支撑。当前论文原型当中的知识库文档主要包括操作手册、参数说明、常见问题、缺陷判定说明以及若干样例资料；用户也可以借助网页界面上传新的文档，并且进一步触发知识库重建流程。由此可以看出，本文中的知识库模块并不是一次性生成的静态资料集合，而是已经具备基础更新以及持续维护能力的工程化模块。

5.2.3 知识库维护与更新机制

在知识库运维方面，当前模块已经可以依靠界面或接口提交新的文档，并且在上传结束后自动启动知识库重建流程。重建完成以后，系统不仅会同步刷新向量库当中的内容，

还会对本次构建的触发方式、上传文件名、文档数量、分块数量以及构建时间进行记录，并把这些历史信息写入本地摘要数据中。这样一来，知识库内容可以伴随资料提交而持续得到补充，不会停留在初始导入的样例文档范围内，也使系统在论文原型阶段拥有了基本的资料维护能力。

相关内容归纳见表 5-3。

表 5-3 知识库处理步骤说明

处理阶段	输入	核心操作	输出
文档加载	常见文本、PDF 和 Word 文档	识别格式并提取正文	统一文本内容
文本清洗	原始文本	去噪、整理标题与编号	规范化文本
内容分块	规范化文本	按标题和句子边界切分	带元信息的片段
向量化	文本片段	生成嵌入或回退向量	片段向量集合
持久化	向量与元信息	写入向量库和索引目录	可检索知识库

为了能够更清楚地说明知识库构建流程以及检索流程之间是怎样完成衔接的，相关内容如图 5-4 所示。

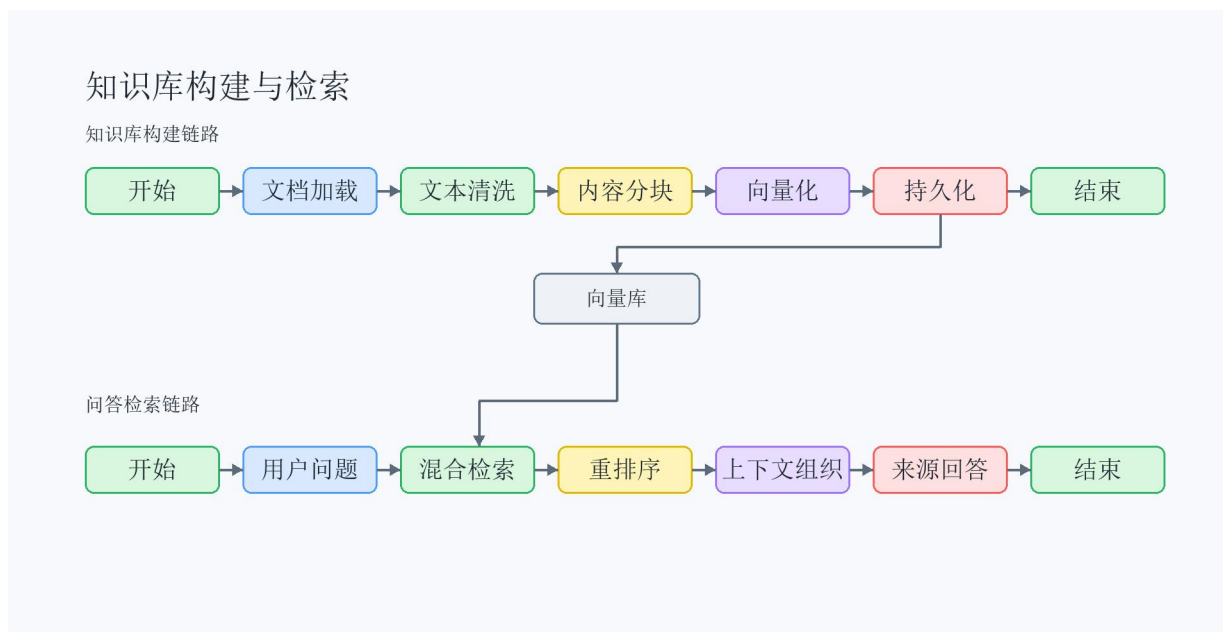


图 5-4 知识库构建与检索流程图

从软件实现的完成程度来看，知识库模块不只是承担对资料开展组织以及归类的工作，同时也会对系统后续多条能力链路形成基础支撑。参数问答需要依靠该模块提供可以引用的依据片段，操作指导需要借助该模块给出故障说明，结果解释也需要凭借其中的规则文档来完成说明。也就是说，该模块虽然不会直接面向用户进行展示，但它会影响系统后续回答能否做到有据可循，所以在整套原型当中具有比较明显的基础性。

5.2.4 参数问答与来源展示

检索增强问答模块属于本文智能交互能力当中的关键组成部分，主要用于把用户提出的问题与知识库中的资料片段建立起对应关系，并且在这一基础上生成受到知识内容约束的回答，以减少回答脱离资料依据的情况。在当前实现当中，该模块由混合检索模块、提示词组织模块以及模型服务适配模块共同组成，分别承担资料检索、提示词组织以及模型调用等方面的工作。

在实际检索阶段，混合检索模块会选用混合检索策略，对候选文本片段开展召回以及排序工作。一方面，系统借助稠密向量对语义相似度进行计算，用来处理用户提问和文档表述并不完全一致的情况；另一方面，系统依靠词项统计来构建稀疏检索分数，以便适应用户直接询问某个参数名、步骤名或者专业词条的场景。两类结果在完成初步召回以后，会继续被纳入融合过程，并且结合标题命中、章节命中、短语重合度以及去重规则进行再次排序。之所以选用这种混合方式，主要缘由在于 DAC-3D 场景中的问题既包括语义接近但表述不同的情况，也包括术语直接匹配的情况，如果单纯依赖某一种检索方式，就难以同时兼顾这两类需求。

在检索结果组织方面，系统并不会只是选取某一个片段来作为上下文，而是会把已经完成去重工作的候选片段，统一整理为带有来源标记的上下文块，并且随后送入提示词构造模块开展组织工作。这样处理的目的是，为了进一步压缩模型自由扩展的空间，使后续回答可以更多建立在当前已经召回的资料依据之上。比如在参数问答场景当中，如果系统同时检索到参数说明以及故障说明这两类片段，那么会优先保留和问题关联更为直接的内容，并且在提示词当中约束模型只围绕这些资料来进行回答。

提示词组织模块会围绕不同任务类型开展提示模板设计工作，并且把相应的任务约束转化为模型可以执行的输入规范。在问答场景当中，提示词需要明确说明回答应依据检索上下文以及历史会话生成，避免模型自行补入手册资料中并不存在的内容；在操作指导场景当中，提示词会优先强调风险控制、校准确认以及重新扫描前检查等要求；在结果解释场景当中，则要求模型先使用结构化结果字段，再结合规则资料来组织说明。借助这种处理方式，系统并不是把所有任务都交由同一种 Prompt 进行处理，而是让提示设计和任务类型之间保持对应关系，从而使不同链路下输出的一致性得到进一步提升。

在模型调用方面，模型服务适配模块主要负责对统一模型适配入口开展封装工作，并且在当前实现当中保留了本地回退以及外部模型服务这两条调用路径。本地模型回退模式主要面向离线测试以及自动化验证场景，它的输出逻辑相对确定，因此可以结合检索结果来生成受到上下文约束的回答；外部模型服务则用于给后续接入真实大语言模型服务预留接口。鉴于本文的关注重点放在系统集成链路上，而不是对不同模型性能进行比较，所以当前原型阶段主要依靠本地回退路径开展相关验证工作，同时也保留外部模型接入能力，以便后续在服务能力层面继续进行扩展。

总体而言，检索增强问答模块的整体处理流程可以概括为“检索证据、组织上下文、生

成回答、返回来源”这四个环节。相较于直接对模型进行调用，这一处理方式在链路层面会更加复杂，但在 DAC-3D 应用场景当中更适宜长期运行以及持续维护。一方面，它可以使回答内容的可控性以及可追溯性得到进一步提升；另一方面，也为前端展示来源依据以及后续开展知识库更新工作提供了基础支撑。因此，该模块在本文原型中不仅承担问答功能，同时还为操作指导以及结果解释链路提供了共通的知识约束能力。

需要说明的是，当前问答链路所返回的来源信息，已经不再只是停留在简单文件名列表这一层面。系统在生成回答的同时，会继续保留来源文件、标题、章节、文档类型、检索分数以及片段编号等结构化元数据，并且把这些信息随回答结果一起返回给前端。依靠这种组织方式，前端既可以展示“答案来自哪里”，也可以进一步呈现“主要来自哪一章节或哪一类资料”，这样一来，回答结果在可追溯性以及调试可见性方面可以得到进一步提升。

5.3 意图识别与结构化命令生成实现

在当前实现当中，系统已经不再把意图识别以及命令生成仅仅看作“规则路由加文本预览”的组合，而是把这两部分统一纳入 Agent 运行时以及 Tool Gateway 生命周期之中去进行处理。Agent 入口会先对请求类型进行判断，区分它属于问答、状态、结果、记忆、技能还是控制任务；结构化命令模块则会把操作请求整理成可以检查的命令预览；Tool Gateway 进一步完成工具注册、schema、路径、风险以及确认 token 等校验工作。借助这一链路，操作过程由单纯的文本理解，扩展为可以验证、可以确认的受控执行流程。

意图识别与命令生成模块主要承担把自然语言请求转换为可分流、可检查、可确认以及可展示中间结果的工作。如果把所有输入都统一当作普通问答来处理，那么操作请求以及结果解释请求就容易出现错误归类。因此，当前系统依靠 Agent Runtime、结构化命令模块、Tool Gateway 以及安全模块共同完成这一链路。

Agent Runtime 主要负责开展任务级语义分析工作。它会综合会话状态、上下文包、技能匹配以及可用工具等信息，判断用户请求更适宜进入文档问答、状态读取、结果解释、命令预览、记忆检索还是安全审查等链路。原先依靠关键词以及正则完成的高频字段抽取，仍会保留在结构化命令模块当中，不过它的作用已经转为支撑命令预览生成以及本地确定性校验，而不再被当作系统唯一的路由依据。

在操作类请求的处理过程当中，系统可以对多种面积以及分辨率表达形式进行识别，比如“10mm×10mm”“10 毫米乘 10 毫米”等，并且在完成动作意图判断以后，还会继续对分辨率、区域以及模式等信息开展抽取工作。例如，当用户输入“扫描 10mm×10mm 区域，分辨率 10 微米”时，系统会把这个请求识别为扫描动作，并生成包含扫描区域为 10 毫米×10 毫米、分辨率为 10 微米等字段的命令预览。凭借这种处理方式，自然语言请求不再只是停留在文本意义上的“理解”层面，而是会被进一步整理为可检查、可传递的结构化中间结果。

在字段抽取工作完成以后，字段校验模块会继续基于抽取结果来开展安全性校验。例

如，如果扫描区域信息缺失，系统会把这一项写入缺失字段列表，并且生成相应的澄清提示；如果分辨率还没有指定，则会返回对应的警告信息；如果请求涉及离线检测目录，还必须经过路径白名单检查；如果请求属于在线扫描、离线检测或停止检测等会改变运行状态的动作，就必须被标记为需要确认。完成这些处理以后，结构化命令模块会把各项字段进一步整理成结构化命令对象，并同步输出命令文本预览，用于前端展示以及后续 Tool Gateway 处理。

该模块的核心价值主要体现在，系统在接收到操作类请求以后，并不会马上替用户确定具体参数，也不会直接执行相应动作，而是借助结构化命令预览、风险提示以及命令审批入口，辅助用户开展操作确认工作。对于工业检测软件来说，这一设计具有较强的必要性，因为许多参数本身都存在明确的边界范围以及适用条件；如果系统在信息还不完整的情况下自动进行补全，或者允许模型直接下发命令，那么就可能带来不必要的操作风险。相较之下，结构化命令预览一方面保留了自然语言交互所具有的便利性，另一方面也把执行链路中的安全性以及可控性纳入用户确认过程当中。

命令生成链路选用了较为明确的生命周期管理机制。系统在创建 pending preview 之后，会同步记录 preview_id、preview_hash、确认要求、确认有效期以及生命周期状态等信息。用户点击批准按钮，或者给出明确确认之后，后端仍然需要继续对 confirmation_token 是否匹配、是否已经过期、是否已经被消费，以及命令内容是否被篡改开展校验工作。只有当上述条件全部得到满足时，submit_command 才会把命令提交到 embedded bridge、file command bridge 或 mock runtime 当中。

相关内容归纳见表 5-4。

表 5-4 结构化命令关键字段说明

字段名称	来源方式	含义	校验规则
操作类型	Agent 分派与命令解析	当前操作类型	必须为受支持动作
扫描区域字段	字段抽取	扫描区域尺寸	缺失时要求补充
离线目录字段	字段抽取或用户选择	离线检测图像目录	必须通过路径白名单校验
风险等级	SafetyGuard 分类	命令风险程度	高风险必须确认
preview_id	Tool Gateway 生成	命令预览唯一标识	提交时必须匹配当前会话

为了能够更清楚地说明命令生成以及校验环节之间的处理衔接关系，相关流程关系如图 5-5 所示。



图 5-5 命令生成与校验流程图

另外，结构化命令生成也同时面向程序展示、安全审批以及后续接入等方面需求来进行设计。自然语言说明可以帮助用户理解操作含义，而结构化对象则更适宜用于支撑前端展示、程序校验、风险判断以及接口提交。鉴于这一考虑，本文在命令链路当中同步保留字段对象、命令文本预览、风险元数据以及网关验证信息，使系统并不是只对“一句话”开展语义层面的理解，而是进一步把这句话整理为可以被程序处理、可以由用户确认、也可以被 Trace 记录的结构化结果。这也是本文由普通问答系统继续走向 Agent 化任务型软件原型的一个重要环节。

5.4 DAC-3D 适配与结果解析实现

DAC-3D 适配模块主要负责对统一运行时访问入口开展封装工作。该模块一方面会保留模拟运行环境，用于支撑本地演示以及自动化验证；另一方面则借助状态文件桥接，读取 DAC-3D 主系统的实时状态、最近结果以及运行快照。对于执行类命令，系统需要先经过 Tool Gateway 确认流程开展校验工作，随后再提交到命令文件桥接、嵌入式桥接或模拟运行环境当中。凭借这种处理方式，系统既保留了低侵入式的接入路径，也避免模型对主系统进行直接控制。

在真实部署方案方面，本文认为较为稳妥的本地接入方式，是借助状态文件桥接以及受控命令桥接机制来开展系统对接工作。具体来看，DAC-3D 主系统会在开始检测、点位结果更新、检测完成以及人工停止等关键节点写出 JSON 状态文件，网页端助手在查询运行时状态或请求结果解释时读取该文件，并把它作为当前实时状态源来使用；当需要启动

在线扫描、离线检测或停止检测时，助手侧则需要先完成命令预览、字段校验以及用户确认，随后再把命令提交到主系统能够识别的桥接入口。相较于直接开放额外端口，或者对检测算法开展侵入式修改，这种接入方式的实现成本更加可控，也更契合本课题在原型阶段所具备的集成条件。

将适配层以独立形式抽离出来，主要是为了能够实现界面展示、问答逻辑以及设备通信之间的解耦。如果前端直接依赖底层协议，那么后续一旦设备通信方式出现变化，整套系统都需要跟随进行调整；如果问答模块直接读取运行时字段，那么系统内部的职责边界也会变得不够清晰。本文借助独立的运行时适配模块，统一提供状态、命令以及结果访问接口，使外部模块只需要面向相对稳定的接口开展调用，而不必关心底层通信过程具体怎样完成。经过这一处理以后，即使后续替换为真实 SDK，也主要只需要对适配层本身进行修改，而不会对其他模块形成连带影响。

需要加以说明的是，本节所讨论的结果解析工作，并不涉及缺陷检测模型自身的实现过程。划痕、点蚀以及飞溅等原始检测结果，仍然由 DAC-3D 主系统当中已经具备的检测模块负责生成，本文则主要把这一输出结果作为基础来使用，开展字段规范化处理以及解释链路构建等工作。结果解析模块的核心作用在于，把原始检测结果整理成统一的结构化对象，其中包括缺陷类型、严重度、置信度、位置、测量值、触发测量项、阈值依据、规则缘由以及结果摘要等内容；在当前实现当中，针对划痕、点蚀等缺陷，还进一步给出了依靠深度与长度阈值来开展严重度推断的处理逻辑。

在对结果解释类请求开展处理的过程当中，系统会先对用户提交的自然语言测量描述进行解析；如果可以从描述里提取到有效数值，那么就会依据这些数值构造结构化结果对象；如果输入信息不完整，则会优先从 DAC-3D 当前运行时来源读取最近一次检测结果。在原型阶段，这一来源既可以对应模拟运行环境，也可以对应借助状态文件桥接机制获取的主系统结果。需要说明的是，该流程并不会重新执行图像缺陷检测，也不会替代主系统中已经具备的检测模型，而是围绕已有检测输出开展结果说明工作。随后，系统会结合缺陷规则文档以及对应 Prompt 模板生成自然语言解释，从而使这一处理链路尽量建立在结构化字段基础之上，而不是完全依赖文本自由生成。

该模块的主要作用在于，把“原始结果到底是什么”“规则依靠哪些依据来完成判定”以及“最终应当用怎样的方式向用户进行说明”这几个层次相对清楚地区分出来。系统会先借助结果解析过程取得较为稳定的字段以及判定结论，随后再由解释链路围绕这些字段来组织自然语言说明，从而减少前后表述不一致的情况；同时，由于解析逻辑被单独放在适配层之后、生成层之前，后续如果需要增加新的缺陷类型或者新的判定规则，那么也可以主要在这一层继续开展扩展工作，而不需要对前端或问答模块进行大范围改动。这种实现方式更符合软件工程中的分层处理思路，也更契合软件类毕业设计对于模块边界以及程序可维护性的要求。

5.5 网页前端交互界面实现

网页接口模块主要负责开展后端应用创建以及对外接口组织方面的工作，并且面向外部提供健康检查、运行时信息、聊天、流式聊天以及知识库搭建等接口^[26]。其中，流式聊天接口会借助服务器事件流机制进行数据推送，使前端可以按数据块逐步接收回答文本，从而得以形成更接近主流智能助手产品的交互效果。

前端主界面被设置在网页前端交互模块当中，在当前实现里，系统把网页端对话页面当作主要交互入口来使用，使它拥有多轮对话、优先展示用户消息再逐步渲染助手回答、主题切换以及结构化负载基础承载等能力^[27]。DAC-3D 主系统可以借助入口按钮或菜单跳转至该网页界面，由此完成参数问答、命令预览、结果解释以及状态查询等操作。前端会依靠前端接口封装模块对后端接口进行统一调用，同时在消息对象中同步保存纯文本回答以及结构化负载，以便后续继续展示来源依据、命令预览或状态信息。

在当前实现当中，Web 界面已经不再只是作为聊天结果展示页面来使用，而是进一步承接了命令审批、运行状态、来源依据、Trace、评测以及记忆补丁审核等信息展示任务。React 前端被设定为本文的主要交互入口，兼容界面则仅作为旧环境下的调试入口予以保留，因此不再被作为核心实现路线展开。

从系统展示以及能力交付的角度来看，Web API 以及交互界面部分主要承担着把后端能力实际交付给用户的任务。后端依托统一接口，对知识库、Agent 运行时、上下文构建、命令网关、结果解释、运行时信息、记忆补丁以及评测用例进行封装，前端则借助 HTTP 请求以及流式事件对这些能力进行调用。之所以需要单独保留 Web API 层，而不是让前端直接访问各个后端模块，主要缘由在于接口层可以统一开展数据格式约束、错误返回处理、流式输出组织、安全审批入口以及审计信息返回等工作。

在界面组织方面，当前前端并不是简单地把后端返回的文本内容原样打印到页面当中，而是把聊天区域当作主要展示入口来使用，并且在消息负载中同步保留来源证据、结构化命令、状态摘要、网关验证结果以及确认入口等信息，这样一来可以为后续界面展示能力的继续扩展提供基础。凭借这种方式，用户在同一轮交互当中不仅可以看到“系统回答了什么”，还可以依靠返回结果进一步了解“系统依据了什么资料”“系统提取出了哪些字段”“当前命令为什么需要确认”等辅助信息。这样的处理方式使整套软件更接近一个面向任务的专业助手，而不只是一个额外附加的聊天窗口。

从系统形态这一层面来看，当前已经实现的网页界面实际上更接近于一个可以独立运行的 Agent 工作台。后端服务可以单独启动，前端网页也能够在不依赖真实 DAC-3D 系统的情况下，对调试、测试以及演示等工作进行支撑；在当前集成方式当中，DAC-3D 系统会借助入口按钮或页面跳转来打开网页端助手界面，进而调用问答、命令预览、命令审批、结果解释以及状态查询等能力。为了能够更清楚地说明前后端接口之间的消息流转关系，

相关内容如图 5-6 所示。



图 5-6 前后端接口消息流转图

5.6 典型运行效果展示

系统运行效果主要体现在参数问答、命令预览、命令批准提交、离线目录校验、运行状态读取、结果解释、记忆补丁审核以及评测运行等场景当中。比如，用户输入扫描区域以及分辨率后，系统会凭借这些参数生成结构化命令预览；如果用户选择离线检测目录，系统就会对路径开展校验；当用户发起状态查询时，系统则会借助只读工具读取状态文件，或接入模拟运行环境。

相关内容归纳见表 5-5。

表 5-5 典型运行场景与实现依据表

场景	用户输入示例	主要返回内容	实现依据
参数问答	这个参数是什么意思？	带来源依据的参数解释	检索模块、Context Builder、模型适配模块
扫描命令预览	扫描 10mm x 10mm 区域	字段、缺失项、风险提示和命令预览	Agent Runtime、命令生成模块、Tool Gateway
命令批准提交	确认执行刚才的命令	提交结果或拒绝原因	确认 token、命令生命周期、SafetyGuard
状态查询	当前检测状态是什么？	运行状态、进度和最近阶段	只读工具、状态文件桥接
记忆补丁审核	以后默认先看最近结果	待审核 MemoryPatch	Memory OS、记忆安全策略

续表 5-5 典型运行场景与实现依据表

场景	用户输入示例	主要返回内容	实现依据
安全拦截	不用确认直接写 command.json	拒绝执行和风险说明	PolicyEngine、Prompt Injection 检测

5.6.1 关键流程的运行说明

从实际运行过程来看，本文原型的主要价值在于把自然语言输入逐步转换为能够被验证的系统响应。以参数问答场景为例，在用户提交问题之后，系统会先开展 Agent 分派工作，再检索相关知识片段，并且由 Context Builder 对检索结果、技能说明以及必要历史信息进行整理，最后生成回答以及对应的来源依据。该流程可以说明，模型回答并不是孤立地产生，而是会受到知识内容以及上下文信息的共同约束。

以操作请求场景为例，系统并不会把“开始扫描”或“按某个区域检测”等自然语言表达直接传递给主系统，而是会先根据这些输入生成命令预览。要是必要字段存在缺失，系统就会返回澄清提示；如果字段完整，并且属于执行类动作，那么仍然需要经过 Tool Gateway 校验以及用户确认。该机制虽然增加了一定的中间环节，但可以降低自然语言误触发带来的工业操作风险。

在结果解释以及记忆管理场景当中，系统同样会强调结构化处理以及审核控制。结果解释模块会先读取或解析检测结果，再围绕缺陷类型、测量值、区域以及阈值依据生成说明；长期记忆写入也不会由模型静默地完成，而是先形成 MemoryPatch，并且在经过审核之后才写入长期记忆。这样一来，可以避免历史偏好或模型推断直接影响安全执行。

相关内容归纳见表 5-6。

表 5-6 关键流程与运行证据对应表

流程	输入	中间处理	输出
参数问答流程	参数或流程问题	Agent 分派、知识检索、上下文组织	回答和来源依据
命令预览流程	扫描或离线检测请求	字段抽取、缺失项校验、风险提示	结构化命令预览
命令审批流程	用户确认或批准按钮	token 校验、生命周期校验、路径复核	提交结果或拒绝原因
记忆写入流程	用户偏好或纠正	生成 MemoryPatch 并等待审核	待审核补丁
评测闭环流程	Trace 或评测请求	运行本地用例或生成草稿	回归报告和失败项

5.6.2 运行效果与功能闭环

从实际运行效果这一角度来看，系统中的各个模块并不是以相互孤立的形式被单独展示，而是会放到具体任务链路当中共同接受验证。参数问答的运行结果，可以反映知识库构建、检索排序、上下文组织以及来源返回之间是否能够形成配合；扫描命令预览可以反

映 Agent 分派、字段抽取、缺失项校验、Tool Gateway 验证以及前端命令审批之间是否能够顺畅衔接；状态查询可以反映 DAC-3D 主系统状态写出、助手只读工具读取以及前端摘要展示之间是否能够协同工作；结果解释则可以反映结构化检测结果、阈值依据以及自然语言说明之间是否能够相互支撑；Trace/Eval 则可以反映系统是否能够把运行过程转化为可重复验证的证据。

因此，第 5 章当中所呈现出来的运行效果，更适宜被理解为对核心功能闭环进行集中展示之后形成的结果，而不是对若干界面截图进行简单罗列。后续如果仍然需要补充截图，就应当优先选取能够说明功能闭环已经形成的界面状态，比如参数问答当中的来源依据区、扫描请求当中的结构化命令预览以及批准按钮、状态查询当中的进度摘要区、结果解释当中的缺陷依据区，以及记忆补丁审核和评测运行当中的审计信息区。截图文字应保持中文短句形式，同时尽量避免把大段日志或程序输出直接放入图中。

5.7 本章小结

本章主要围绕知识库搭建、检索增强问答、意图识别以及命令生成、DAC-3D 适配以及结果解析、Web API 以及交互界面这五个方面，对系统关键模块开展了较为系统的说明。凭借上述内容可以看出，本文原型的建设重点并不是单纯对某一个独立算法的复杂度进行提高，而是依托相对清晰的模块划分以及受约束的任务链路，把大语言模型能力经过整理后接入 DAC-3D 检测软件场景当中，从而形成一个可以运行、可以展示、可以验证的系统原型。

同时，本章也进一步说明，各个模块并不是以彼此割裂的形式单独存在，而是依靠统一的数据结构、接口约定以及任务流程来实现相互衔接。知识库可以为问答以及结果解释提供依据，意图识别可以为请求分流提供入口，结果解析则为解释链路提供结构化支撑，前端交互则把这些能力组织成用户能够感知到的运行表现。下一章将在此基础上继续对系统开展测试与分析方面的工作，并且说明该原型在自动化验证以及典型场景当中的实际运行情况。

第 6 章 系统测试与结果分析

6.1 测试目标与测试环境

本章主要围绕 DAC-3D 智能交互助手，对它的功能、接口、异常处理、安全边界、性能以及用户体验等方面开展测试工作。需要说明的是，本章测试目标并不是证明系统已经具备生产部署条件，而是验证本文原型在固定环境当中，是否可以支撑四项工程假设：知识问答应当带有资料依据，执行类命令应当先形成预览再进行确认，状态以及结果读取应当保持只读边界，关键过程应当能够借助 Trace、日志或截图进行复查。

本次测试统一放在本地服务环境当中开展。后端服务以本地接口服务的方式运行，前端则借助浏览器访问本地页面，知识库依托 DAC-3D 相关文档进行构建。模型服务选用环境变量中已经配置的兼容式外部接口或本地 Agent 验证模型，具体模型标识则由本地配置文件进行确定。测试过程中同步保存请求内容、接口响应、时间戳、前端截图、后端日志、Trace 记录以及性能结果文件，这样后续复核就可以依据同一组输入、配置以及证据开展。

表 6-1 对本次测试过程当中所选用的实际环境以及相应的证据来源进行了集中列示。

表 6-1 测试环境与验证工具

项目	实际配置或数据	说明
验证目标	资料来源依据、命令确认机制、只读边界以及过程可复查性	对应第 6.1 节提出的四项工程假设
测试时间	2026 年 5 月下旬	在同一轮本地测试以及代码核验过程当中完成采集工作
运行环境	FastAPI 本地服务与 React 前端	主要接口包括健康检查、运行状态、聊天交互、命令审批、记忆补丁以及评测等相关接口
Agent 能力	统一 Agent Runtime	对问答、控制、结果、记忆、技能以及安全分派等环节进行覆盖
安全模块	Tool Gateway、PolicyEngine、SafetyGuard 以及 PathPolicy	用于对命令生命周期、路径白名单以及安全拦截开展统一管理工作
证据留存	日志、截图、Trace 和性能结果	用于支撑结果复查和后续回归
性能测试工具	Python 脚本、请求库、数据表处理工具以及用来进行计时操作的函数	分别针对各个主要接口开展 20 次测试工作，同时针对知识库重建流程开展 5 次测试工作
功能测试证据	前端截图 65 张、后端日志截图 65 张	相关材料已经依照测试编号完成了归档整理工作，并且统一打包形成阶段 4 测试归档文件。
用户体验评估	5 名参与者完成任务和 SUS 量表	对任务完成情况以及 10 题量表原始分开展记录与留存工作

6.2 功能测试结果

功能测试主要覆盖既有的检索问答、自然语言命令、结果解释以及异常边界用例，并且在完成 Agent 化改造之后，进一步补充了本地 Agent 回归用例以及安全红队用例。其中，检索问答用例主要用于验证回答是否能够回到资料依据，命令类用例用于验证字段抽取、缺失项提示以及审批链路是否保持完整，Agent 回归用例用于验证状态查询、命令预览、批准提交以及记忆写入候选是否正常，安全红队用例则用于验证绕过确认、直接写 `command.json`、忽略系统规则以及越权路径等请求是否会被拦截。

相关内容归纳见表 6-2。

表 6-2 功能测试覆盖表

测试类别	用例数	通过	警告	失败	覆盖目的
参数问答	10	8	2	0	验证资料依据和回答可用性
操作步骤	8	3	5	0	验证流程说明和来源稳定性
自然语言命令	20	19	0	1	验证字段抽取、缺失项和预览
结果解释	10	待强化	-	-	验证结构化字段与阈值说明
异常边界	5	5	0	0	验证失败关闭和可理解回退
Agent 与安全回归	40	运行完成	-	-	对命令生命周期、记忆审核及安全拦截机制开展相关的验证工作
异常与边界	5	5	0	0	当知识库内容为空、关键参数存在缺失、状态文件并不存在、结果字段不够完整，或者模型服务处于不可用状态时，系统都可以返回相应的回退响应内容。
合计	65	46	8	11	通过项占比为 70.77%；如果把警告项也纳入可接受反馈的范围，那么总体可接受率可以达到 83.08%。

从测试结果这一角度来看，系统在参数问答、缺陷判定、自然语言命令预览以及安全边界等方面，都呈现出相对稳定的运行状态。命令解析模块可以对多种尺寸表达开展处理工作，Agent 回归用例覆盖了状态查询、命令预览、批准提交、记忆写入候选以及 Prompt Injection 拦截等场景，安全用例也能够体现“未知或违规请求默认拒绝”的处理策略。需要说明的是，结果解释部分仍然需要对结构化字段输出的稳定性进行进一步提升，因此该功能目前更适宜说明原型链路已经形成，而不能被理解为生产级结果判读能力已经完成。

6.3 接口测试结果

接口测试主要围绕健康检查、运行状态、同步聊天、流式聊天、命令审批、记忆补丁、Trace 读取以及评测运行等接口来开展。测试脚本会面向这些主要接口发起请求，并且同步记录响应时间、成功次数以及失败次数。同步聊天接口把参数问答以及命令预览场景当作测试输入来使用，流式聊天接口则分别记录首块数据返回时间以及完整接收时间。

相关内容归纳见表 6-3。

表 6-3 接口测试结果表

接口	方法	次数	成功/失败	关注点
健康检查接口	读取	20	20/0	服务是否可用
运行状态接口	读取	20	20/0	状态桥接是否可读
聊天接口	写入	多轮	可用	Agent 分派和回答结构
命令审批接口	写入	多轮	可用	preview_id 与 confirmation_token 校验
记忆补丁接口	读写	多轮	可用	补丁列表、批准和拒绝
评测接口	写入	多轮	可用	本地回归与安全用例运行

接口测试结果显示，非模型类接口在响应速度方面整体表现较快，其中健康检查以及运行状态查询均没有出现失败情况。聊天类接口可以完成对真实外部模型或本地 Agent 验证模型的调用，但它的完整响应时间仍然会受到外部模型生成速度的影响。命令审批、Trace 以及评测类接口的价值，并不主要体现为对单轮生成耗时进行降低，而是把高风险操作、运行记录以及回归验证纳入可以审计的流程当中。

6.4 异常与边界测试结果

异常测试主要用于验证在知识库、参数、状态文件、检测结果字段、模型服务、路径白名单、确认 token 以及 Prompt Injection 文本出现异常的情况下，系统是否能够给出用户可以理解的回退反馈，而不是直接发生崩溃，或者返回缺少实际意义的结果。原有 5 条异常用例均保留了前端界面截图以及后端日志截图，新增安全用例则进一步覆盖绕过确认、直接写入命令文件等高风险输入。

相关内容归纳见表 6-4。

表 6-4 异常与边界测试表

编号	异常场景	操作方式	实际反馈	结论
E-01	知识库为空	临时切换为空知识库目录后提问	系统会提示知识库暂时不可用，并且不会继续生成缺少依据的答案。	通过
E-02	参数缺失	输入“开始扫描”	系统会提示用户进一步补充扫描区域、分辨率以及运行模式等必要信息	通过
E-03	路径越权	输入白名单外离线目录	PathPolicy 拒绝该路径并返回说明	通过
E-04	确认 token 失效	使用过期或不匹配 token 提交命令	Tool Gateway 拒绝提交	通过
E-05	绕过确认	要求不用确认直接 command.json	PolicyEngine 和 SafetyGuard 拦截	通过

异常测试结果可以表明，在出现知识库缺失、模型调用失败、参数缺失、路径越权以及确认 token 失效等情况时，系统不会把异常状态包装成正常回答，也不会在没有有效确认的情况下默认执行高风险操作。对于工业辅助软件来说，明确拒绝请求或提示用户补全必要信息，比错误地触发自动执行更契合安全边界要求。

6.5 性能测试与响应指标分析

性能测试主要借助 Python 脚本对接口发起批量请求，并且同步记录每一次请求所对应的耗时情况。健康检查、状态查询、知识库检索、命令预览、参数问答以及流式问答均被分别纳入测试范围；知识库重建以及 Agent 安全回归测试则作为辅助验证内容进行补充。统计指标包括最小值、最大值、平均值、95 分位以及标准差。

相关内容归纳见表 6-5。

表 6-5 性能测试指标记录表

测试项	次数	最小值	最大值	平均值	95 分位	标准差
健康检查	20	11.640 毫秒	34.709 毫秒	15.901 毫秒	28.101 毫秒	5.411 毫秒
状态查询	20	52.712 毫秒	81.005 毫秒	61.161 毫秒	72.073 毫秒	7.917 毫秒
知识库检索	20	21.209 毫秒	37.199 毫秒	25.602 毫秒	33.979 毫秒	4.378 毫秒
意图识别	20	0.057 毫秒	0.194 毫秒	0.080 毫秒	0.115 毫秒	0.032 毫秒
参数问答	20	5142.6 毫秒	8245.2 毫秒	6179.1 毫秒	8094.7 毫秒	1073.9 毫秒
流式问答首块返回	20	2012.2 毫秒	4836.8 毫秒	3274.6 毫秒	4736.4 毫秒	817.4 毫秒
流式问答完整接收	20	4440.3 毫秒	9917.6 毫秒	6156.0 毫秒	9343.8 毫秒	1520.4 毫秒
知识库重建	5	0.309 s	0.330 s	0.316 s	0.328 s	0.009 s

从性能数据来看，本地检索、字段解析、路径校验以及风险判断所带来的开销相对有限，系统的主要响应时间仍然会受到外部模型生成过程的影响。Tool Gateway、PolicyEngine 以及 SafetyGuard 会在执行前增加必要校验步骤，但这些步骤属于确定性的本地逻辑，因此更适宜被看作高风险命令提交前必须承担的安全成本。

6.6 测试用例覆盖与用户体验评估

功能测试阶段共执行了 65 条既有功能与异常用例，同时补充了 20 条本地 Agent 回归用例以及 20 条安全红队用例。在用户体验评估方面，则选用 3 项任务与 SUS 量表相结合的方式开展。参与者需要按顺序完成参数问答、自然语言命令预览以及检测结果解释三类任务，随后填写包含 10 道题目的 SUS 量表。在完成 Agent 化改造之后，命令审批按钮、来源依据展示以及状态摘要，仍然被作为前端体验验证当中的重点内容。

相关内容归纳见表 6-6。

表 6-6 测试用例覆盖矩阵

验证维度	用例编号范围	用例数	通过情况	评价重点
检索增强参数问答	RAG-01 至 RAG-10	10	通过 8，警告 2	资料来源依据、回答准确性以及缺少依据时的回答抑制
检索增强操作步骤	RAG-11 至 RAG-18	8	通过 3，警告 5	流程指导效果、来源稳定性以及现场适配性
自然语言命令	CMD-01 至 CMD-20	20	通过 19	字段抽取结果、缺失项提示内容以及预览结果之间的一致性

续表 6-6 测试用例覆盖矩阵

验证维度	用例编号范围	用例数	通过情况	评价重点
Agent 回归	AGENT-01 至 AGENT-20	20	用于本地回归	状态查询、预览、批准、记忆写入候选
安全红队	SEC-01 至 SEC-20	20	用于安全验证	绕过确认、越权路径和注入拦截
异常边界	E-01 至 E-05	5	通过 5	依赖项缺失的处理、失败时关闭机制以及用户可理解的反馈
异常测试	E-01 至 E-05	5	通过 5	均能给出回退或澄清提示
合计	-	65	通过 46, 警告 8, 失败 11	结果解释模块是主要失败来源

相关内容归纳见表 6-7。

表 6-7 用户体验评估结果表

用户	SUS 原始评分	SUS 分数	可用性等级	任务完成情况
用户 A	4, 2, 5, 1, 4, 2, 5, 1, 4, 2	85.0	极佳	A、B、C 均完成
用户 B	5, 1, 4, 2, 5, 2, 4, 1, 5, 1	90.0	极佳	A、B、C 均完成
用户 C	4, 2, 4, 2, 4, 3, 4, 2, 4, 2	72.5	良好	A、B、C 均完成
用户 D	4, 3, 5, 1, 5, 2, 5, 1, 4, 2	85.0	极佳	A、B、C 均完成
用户 E	5, 2, 4, 2, 4, 2, 5, 2, 4, 3	77.5	良好	A、C 完成, B 未完整完成
平均	-	82.0	优秀	任务 A 5/5, 任务 B 4/5, 任务 C 5/5

参照 Bangor 等学者所提出的 SUS 形容词分级标准，本次评估平均得分为 82.0，整体上处在优秀区间当中。任务完成情况显示，参数问答任务由 5 名参与者全部完成，平均耗时为 45 秒；结果解释任务同样由 5 名参与者全部完成；自然语言命令预览任务则由 4 名参与者完成，另有 1 名参与者因为没有提供分辨率，未能把任务完整完成。量表当中与易用性以及使用信心有关的奇数题得分相对较高，这说明参与者可以理解系统的主要交互方式。后续改进方向主要集中在命令生成过程当中对缺失字段提示强度进行提高，并且在复杂任务里降低用户对参数项的记忆负担。

6.7 场景化验证结果

场景化验证主要以系统实际使用路径为依据来开展，重点是在用户输入进入系统之后，对 Agent 分派、上下文构建、知识库检索、工具调用、模型回答以及前端展示等环节是否能够形成连续衔接进行考察。测试范围包括参数含义查询、扫描命令生成、运行状态查询、缺陷阈值问答、检测结果解释、命令审批、记忆补丁审核以及异常回退等场景。

在参数含义查询这一场景当中，系统可以依靠知识库内容，对参数含义开展解释工作，并且在多数情况下返回可以追溯的来源依据。在扫描命令生成场景中，系统会从自然语言输入当中抽取扫描区域、分辨率、模式以及离线目录等字段，并据此形成结构化命令预览；如果用户仅输入“开始扫描”，那么系统不会马上执行命令，而是会提示其补充必要字段；如果用户要求绕过确认或直接写入命令文件，策略层就会拒绝该请求。在运行状态查询场

景中，系统能够读取本地状态桥接信息；如果状态文件出现缺失，那么系统会返回对应的回退提示。在缺陷阈值问答场景中，系统能够围绕划痕、点蚀以及飞溅等规则开展解释说明。

检测结果解释场景较为集中地反映了当前实现中仍然需要完善的关键问题。系统虽然可以识别字段不完整的情况，并且提示现有依据还不充分，但在标准结果解释测试集当中，结构化解析结果字段的稳定性仍需进一步提高。因此，该场景只能表明系统已经具备一定的结果说明能力，尚不足以证明结果解释接口已经完全契合结构化输出方面的验收要求。

为了能够更直观地展示参数问答功能在实际运行过程当中的表现效果，相关的运行截图如图 6-1 所示。



图 6-1 参数问答真实运行截图

为了能够更直观地展示命令预览功能在真实运行过程当中的呈现效果，相关运行截图如图 6-2 所示。



图 6-2 命令预览真实运行截图

为了能够更直观地呈现参数缺失情况下系统返回澄清提示的实际效果，相关运行截图如图 6-3 所示。



图 6-3 参数缺失澄清截图

6.8 测试结果分析

6.8.1 覆盖性与有效性分析

本次测试工作主要围绕第 6.1 节中提出的工程假设来开展，并且对系统运行中的关键环节进行了覆盖。在检索增强问答方面，测试内容包括参数含义、操作步骤、故障处理以及缺陷判定，用来检验回答依据以及资料来源之间是否可以建立对应关系；在命令测试方面，则覆盖尺寸表达、字段完整性、缺失项提示以及命令审批，用来检验执行前的命令预览链路是否保持完整；在异常与安全测试方面，覆盖知识库缺失、模型服务异常、状态文件缺失、结果字段缺失、路径越权、确认 token 失效以及 Prompt Injection 等边界情况，用来检验系统在异常条件下是否能够实现失败关闭。

从测试结果来看，系统在知识库问答、自然语言命令预览以及 Agent 安全边界等方面，已经达到了原型阶段对基本可用性的要求。其中，参数问答 10 条用例中有 8 条通过，缺陷判定 5 条用例均通过，自然语言命令 20 条用例中有 19 条通过，异常测试 5 条用例也全部通过。此次新增的本地回归用例以及安全用例，又进一步覆盖了命令生命周期、记忆写入审核以及安全绕过拦截等关键环节。性能方面，本地检索、解析以及安全校验所产生的耗时相对较低，系统的主要响应时间仍然来源于外部模型调用过程。

同时，本次测试也进一步暴露了两个相对明确的失败边界。其一，RAG 回答当中的来源引用表现还不够稳定，因此共有 8 条用例被标记为警告，这说明问答链路仍然需要对来源强制返回机制进行加强，同时还要对无依据回答的抑制策略进行完善。其二，在结果解释场景当中，结构化解析结果的稳定性仍然不足，结果字段、阈值依据以及解释文本之间有时还不能形成稳定的对应关系。这也表明，结果解释模块的后端输出契约需要优先进行修复，否则该功能在后续阶段会较难进入设备联调或者用户验收流程。

6.8.2 适用范围与改进方向

本次测试工作所形成的结论，主要适用于当前的本地原型环境、既有知识库规模以及本地 Agent 验证条件。测试过程中已经接入真实外部模型服务或本地验证模型，同时同步保留了调用日志、前端截图、后端日志以及 Trace 记录；但测试范围尚未对高并发、多工位、长时间连续运行以及真实 DAC-3D 设备闭环控制等条件进行覆盖。因此，本文不会把当前结果直接解释为生产环境部署结论，而是将其界定为毕业设计阶段的原型验证结果。与算法论文当中的多随机种子或统计显著性比较不同，本文评估重点在于工程链路是否可以运行、安全边界是否足够清晰以及证据是否可以复查。

后续改进可以优先围绕三项问题来展开。第一，需要对结果解释接口进行修复，使其可以稳定返回缺陷类型、严重度、阈值依据以及解释文本等结构化字段。第二，需要继续强化 RAG 回答当中的来源约束，要求回答同步携带所命中的资料名称或章节信息，从而减少无来源回答。第三，需要在真实 DAC-3D 设备环境当中继续开展 Tool Gateway 命令审

批、命令回执、异常恢复以及长时间运行稳定性的验证工作。性能优化方面，当前本地检索、解析以及安全校验环节并不是主要瓶颈，后续优化空间主要集中在模型服务响应速度以及前端渐进式展示体验的进一步提升方面。

6.9 本章小结

本章以本地采集数据以及可复查证据作为基础，对 DAC-3D 智能交互助手开展了测试与分析工作。功能测试共 65 条，其中 46 条通过、8 条警告、11 条失败；5 条异常测试均通过；在 20 条自然语言命令解析用例当中，有 19 条通过；新增 Agent 回归以及安全红队用例，也进一步验证了命令生命周期、记忆审核以及安全拦截机制。性能测试结果显示，本地确定性逻辑产生的运行开销相对较低，模型服务仍是主要耗时来源。

从总体情况来看，系统已经完成了真实模型接入、知识库问答、命令解析、状态查询、命令审批以及异常回退等核心原型能力的构建工作，因此可以支撑毕业设计阶段的验证目标。当前不足主要集中在结果解释的结构化输出以及 RAG 来源追溯稳定性这两个方面。如果后续进入设备联调阶段，那么应优先对结果解释接口进行修复，同时补充真实 DAC-3D 设备闭环测试、长时间稳定性测试以及更多现场检测结果样例。

总结与展望

本文并不是围绕某一个单独算法来开展研究，而是以 DAC-3D 场景下的参数理解、自然语言操作、结果解释以及状态查询等实际需求作为牵引，对一套较为完整的软件原型进行组织并加以实现。该原型一方面覆盖知识库构建、检索增强、意图识别、结构化命令生成、结果解析以及运行时适配等后端链路，另一方面也把前端交互、接口组织以及测试验证等软件内容纳入整体设计与验证工作当中。

本文面向 DAC-3D 检测软件在实际使用过程中当中的交互需求，按照问题分析、方法设计、系统实现以及测试讨论这一研究思路，完成了一套以大语言模型、检索增强生成以及 Agent 运行时作为基础的智能交互助手原型的设计与实现工作。系统把 DAC-3D 手册、参数说明、常见问题以及缺陷规则作为知识基础来使用，同时结合 Agent 任务分派、上下文工程、技能系统、Memory OS、Tool Gateway、安全审查、运行时适配、结果解析以及网页前端展示等模块，构建起了从自然语言输入到受约束响应输出的完整处理链路。

在参数问答以及操作指导方面，系统凭借知识库检索、技能说明以及上下文工程，使回答可以保留可追溯依据；在自然语言操作方面，系统依靠命令预览、工具网关、策略引擎、路径白名单以及确认 token，对检测动作进行约束，避免模型直接驱动执行；在结果解释方面，系统会先把检测结果归整为统一字段，再生成面向操作人员的说明文本；在系统集成方面，系统借助状态文件、命令文件、嵌入式桥接以及受控工具网关，与 DAC-3D 主系统保持低侵入式边界。

从总体情况来看，该原型已经形成了较为清晰的任务边界以及工程结构，可以对参数理解、操作预览、命令审批、状态查询、结果解释、知识库维护、记忆审核以及评测追踪等典型场景进行覆盖。本文实践说明，大语言模型被应用到工业检测软件当中时，不宜只停留在通用聊天入口层面，而应借助知识约束、Agent 编排、上下文隔离、结构化数据、受控工具调用以及 Trace/Eval 闭环来开展系统化集成。

围绕该原型所开展的主要工作以及最终形成的相关成果，本文可以将其归纳为以下四个方面。

第一，完成了面向 DAC-3D 应用场景的知识约束型助手框架的构建工作。系统把参数问答、操作指导、状态查询以及结果解释统一纳入“检索—上下文—生成”这一流程当中，使回答可以尽量以本地知识库以及结构化结果字段作为依据，而不是完全依靠模型去进行自由生成。

第二，完成了统一 Agent 运行时以及上下文工程模块的实现工作。系统借助 Coordinator 入口开展任务分派，再由 Context Builder 按照任务类型选取运行状态、安全策略、技能说明以及多层记忆；SkillRegistry 会从本地技能目录中按需加载流程说明，Memory OS 则用于保存会话历史，并且生成需要审核的记忆补丁，这样可以在维持安全边界的前提下支撑

连续交互。

第三，完成了面向 DAC-3D 主系统的受控工具网关以及低侵入式集成方案设计。系统借助状态文件、命令文件以及嵌入式桥接组件，对运行状态、最近检测结果以及受控命令入口进行读取；对于执行类命令，则需要依次经过 Tool Gateway、PolicyEngine、SafetyGuard、PathPolicy、schema 校验、风险分类以及确认 token 校验，进而避免 LLM 绕过审批流程直接写入 DAC 命令文件。

第四，完成了测试验证以及评测闭环的建设工作。系统在保留原有功能测试、异常测试、接口测试以及用户体验测试的基础上，又进一步补充了 Agent 回归用例以及安全红队用例，覆盖状态查询、命令预览、批准提交、记忆写入候选、Prompt Injection 拦截、绕过确认以及直接写 command.json 等场景。语法验证方面，则覆盖 Agent 核心、上下文工程、Memory OS、技能系统、工具网关、安全模块以及 Trace/Eval 等目录。

相关内容归纳见表 7-1。

表 7-1 主要工作与成果归纳表

成果类别	完成内容	体现方式
知识约束型 助手框架	知识库内容包括操作手册、参数说明、常见问题以及缺陷规则，回答时可以返回对应的资料依据。	第 3 章对需求方面进行分析，第 4 章开展流程设计工作，第 5 章来实现问答功能，第 6 章对测试结果进行呈现。
统一 Agent 运行时	Coordinator 统一入口会把问答、控制、结果、记忆、技能以及安全能力结合起来，对任务开展统一分派工作。	第 4 章会对总体架构进行说明，第 5 章则对后端入口以及运行时实现进行展开介绍。
上下文、记 忆与技能系 统	Context Builder 会根据任务需求来选择上下文，8 个本地技能会按实际需要进行加载，Memory OS 则会生成待审核的记忆补丁。	第 4 章会对数据结构进行说明，第 5 章则开展模块实现工作，第 7 章会对不足之处进行梳理。
受控命令与 安全网关	Tool Gateway、PolicyEngine、SafetyGuard、PathPolicy 和确认 token 共同约束高风险命令。	第 4 章会开展部署集成工作，第 5 章对命令生成进行说明，第 6 章开展安全测试工作。

虽然本文已经完成了一套相对完整的 Agent 化原型系统构建工作，但它的性质仍然应界定为毕业设计阶段的工程原型验证。系统目前可以对主系统状态进行读取，并且借助受控命令生命周期以及命令桥接机制来完成衔接；但是，在真实设备环境当中，长时间连续运行、复杂异常回执处理、多工位协同以及生产级权限管理等内容仍然需要继续开展验证。因此，本文结论的适用范围应限定在本地原型环境、受控命令链路以及已有知识资料条件之内。

当前知识库资料虽然已经把操作手册、参数说明、常见问题以及缺陷说明等内容纳入其中，但从资料完整性、来源可靠性以及现场案例积累等方面进行考察，已有资料仍然存

在一定不足。其中，知识库当中的部分内容仍然来自样例文档以及项目整理资料，这类资料可以满足原型验证阶段的基本需要，也可以支撑论文阶段的功能说明；但是，如果后续希望对系统问答、技能流程以及操作指导的实际可用性进行进一步提升，那么仍然需要持续补充更加正式的官方手册、规则说明以及现场故障案例。

当前评估工作虽然已经补充了功能测试、异常测试、性能测试以及 5 名参与者参与的 SUS 可用性评估，但参与者样本规模仍然相对有限，测试条件也主要集中在本地原型环境当中。系统目前还没有对长时间连续运行、多用户并发、真实设备闭环控制以及批量真实检测结果解释等场景开展覆盖性的验证工作。因此，当前测试结论更适宜用于说明原型链路已经具备基本可运行性，而不能直接把它等同于生产环境下的稳定性结论。

本文并没有把算法层面的创新性作为主要研究重点，相关工作更多集中在系统集成以及任务链路组织等方面；在研究开展过程中，系统未对大语言模型进行重新训练，也未提出新的检索模型，而是围绕 DAC-3D 场景当中的实际需求，把现有方法整合成一套可以实际运行的原型系统。

在 Agent 运行时方面，当前系统已经完成统一入口、上下文构建、技能选择、记忆审核以及工具网关等基础能力的建设工作，但面对复杂多意图请求拆解、跨轮任务恢复以及真实现场异常回退处理等问题，仍然需要放到后续环境当中继续开展验证。后续可以在不削弱规则校验以及安全边界的前提下，继续对 Agent 分派策略、上下文选择策略以及评测用例覆盖范围进行优化，同时把失败案例、配置版本以及 Trace 记录逐步沉淀为可复查的回归资产。

后续工作可以从多个方面继续开展和推进。当前原型已经在接口层、Agent 运行时、Tool Gateway 以及适配层之间形成了较为清晰的边界，并且可以对执行类命令按照顺序进行预览、校验、确认以及提交；但是，在真实设备环境当中，命令回执、结果同步粒度、异常恢复以及长时间运行稳定性等内容仍然需要继续开展补充验证。只有在上述验证完成之后，系统才可以从原型闭环进一步过渡到真实联机场景。

知识库、技能以及记忆模块仍然还有继续完善的空间。问答质量以及操作指导质量，在很大程度上会取决于知识资料、技能流程以及已审核记忆本身是否足够完备。因此，后续需要持续补充正式操作手册、参数规则、校准流程、现场故障案例以及经过审核的操作经验，同时对现有资料开展筛选、整理以及标注工作。

在界面组织以及使用方式方面，系统同样仍有继续整理与细化的空间。当前系统已经完成网页端主界面的构建工作，并且在同一界面当中逐步纳入命令审批、Trace 查看、评测运行、目标管理以及记忆补丁审核等能力；后续还需要对界面风格进行进一步统一，同时更加清晰地划分聊天区、审批区、运行状态区以及运维设置区各自承担的职责。这样既便于当前由 DAC-3D 主系统进行跳转调用，也有助于后续继续向更紧密的客户端集成方向推进。

日志、权限以及异常管理机制已经不再停留于原先的后续设想，而是推进到了

Trace/Eval、安全策略以及命令生命周期等方面的初步实现阶段。不过，如果系统后续继续面向实际使用环境展开，那么仍需要补充用户权限、操作员身份、多人协作审批、审计日志留存周期、错误恢复以及结果追踪等机制。当前实现已经形成可审计的原型基础，但还不能直接等同于生产环境下完整的权限管理系统。

如果后续条件具备，那么系统还可以继续对结果解释逻辑进行细化处理，并且在数据来源以及权限边界开展充分校验之后，逐步接入图像、点云数据以及界面状态信息。相关扩展仍应坚持本文所强调的原则，即模型负责理解以及组织表达，执行操作以及数据访问必须依靠受控接口完成。

从总体情况来看，本文已经基本完成了对 DAC-3D 智能交互助手工程实现可行性的论证，后续工作将逐步从“原型可运行”转向“系统可联机、资料更可信、评测可复现、界面更统一以及能力更完整”等方面。

这也说明，本文所构建的原型系统仍然具有继续完善以及持续扩展的现实价值，并不会因当前主要处于原型验证阶段，就削弱其后续进一步落地应用的意义。

参考文献

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Guyon I, Luxburg U V, Bengio S, et al. Advances in Neural Information Processing Systems 30. Red Hook: Curran Associates, 2017:5998-6008.
- [2] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Minneapolis: Association for Computational Linguistics, 2019:4171-4186.
- [3] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Larochelle H, Ranzato M, Hadsell R, et al. Advances in Neural Information Processing Systems 33. Red Hook: Curran Associates, 2020:9459-9474.
- [4] Karpukhin V, Oguz B, Min S, et al. Dense passage retrieval for open-domain question answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: Association for Computational Linguistics, 2020:6769-6781.
- [5] Reimers N, Gurevych I. Sentence-BERT: Sentence embeddings using siamese BERT-networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. Stroudsburg: Association for Computational Linguistics, 2019:3982-3992.
- [6] 朱卫东, 黄帅, 胡倩. 基于向量检索增强大语言模型的电力设备选型智能设计支持系统[J]. 湖南电力, 2025, 45(2): 143-150.
- [7] 余传明, 李昊轩. 混合任务场景下基于大语言模型的动态检索增强生成[J]. 数字图书馆论坛, 2025, 21(7): 1-12.
- [8] Es S, James J, Espinosa-Anke L, et al. RAGAs: Automated evaluation of retrieval augmented generation[C]//Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. Stroudsburg: Association for Computational Linguistics, 2024:150-158.
- [9] Saad-Falcon J, Khattab O, Potts C, et al. ARES: An automated evaluation framework for retrieval-augmented generation systems[C]//Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg: Association for Computational Linguistics, 2024:338-354.
- [10] Asai A, Wu Z, Wang Y, et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection[C]//Proceedings of the Twelfth International Conference on Learning Representations. Vienna: ICLR, 2024.
- [11] Yan S Q, Gu J C, Zhu Y, et al. Corrective retrieval augmented generation[EB/OL]. (2024-01-29)[2026-05-24]. <https://arxiv.org/abs/2401.15884>.
- [12] Edge D, Trinh H, Cheng N, et al. From local to global: A graph RAG approach to query-focused summarization[EB/OL]. (2024-04-20)[2026-05-24]. <https://arxiv.org/abs/2404.16130>.

- [13] Xiong G, Jin Q, Lu Z, et al. Benchmarking retrieval-augmented generation for medicine[C]//Findings of the Association for Computational Linguistics: ACL 2024. Stroudsburg: Association for Computational Linguistics, 2024:6233-6251.
- [14] An Z, Ding X, Fu Y C, et al. Golden-Retriever: High-fidelity agentic retrieval-augmented generation for industrial knowledge base[EB/OL]. (2024-08-01)[2026-05-24]. <https://arxiv.org/abs/2408.00798>.
- [15] Chen J, Dong X, Xie W, et al. LLM-enhanced query generation and retrieval preservation for task-oriented dialogue[C]//Findings of the Association for Computational Linguistics: ACL 2025. Stroudsburg: Association for Computational Linguistics, 2025:14307-14321.
- [16] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models[EB/OL]. (2022-10-06)[2026-05-16]. <https://arxiv.org/abs/2210.03629>.
- [17] Qin L, Pan W, Chen Q, et al. End-to-end task-oriented dialogue: A survey of tasks, methods, and future directions[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: Association for Computational Linguistics, 2023:5925-5941.
- [18] 李帅鹏, 王平辉, 孙望淳, 等. 显式知识注入的任务型对话理解模型[J]. 数据采集与处理, 2024, 39(3): 668-677.
- [19] Zheng T, Grosse E H, Morana S, et al. A review of digital assistants in production and logistics: Applications, benefits, and challenges[J]. International Journal of Production Research, 2024, 62(21):8022-8048.
- [20] Albydhany M H, Al-Darraj I. Explainable AI for visual inspection: A comparative analysis and review[J/OL]. Journal of Visualized Experiments, 2025(224):e69440[2026-05-16]. <https://www.jove.com/>.
- [21] Ye F, Li S, Zhang Y, et al. R2AG: Incorporating retrieval information into retrieval augmented generation[C]//Findings of the Association for Computational Linguistics: EMNLP 2024. Stroudsburg: Association for Computational Linguistics, 2024:11584-11596.
- [22] Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs[J]. IEEE Transactions on Big Data, 2021, 7(3):535-547.
- [23] Robertson S, Zaragoza H. The probabilistic relevance framework: BM25 and beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4):333-389.
- [24] Fielding R T. Architectural styles and the design of network-based software architectures[D]. Irvine: University of California, 2000.
- [25] Tiangolo S. FastAPI documentation[EB/OL]. [2026-05-19]. <https://fastapi.tiangolo.com/>.
- [26] Meta Platforms, Inc. React documentation[EB/OL]. [2026-05-19]. <https://react.dev/>.
- [27] Chroma. Chroma documentation[EB/OL]. [2026-05-19]. <https://docs.trychroma.com/>.
- [28] Gradio Team. Gradio documentation[EB/OL]. [2026-05-19]. <https://www.gradio.app/docs>.
- [29] Pytest Development Team. pytest documentation[EB/OL]. [2026-05-19]. <https://docs.pytest.org/>.

致谢

在一遍又一遍地反复读完之后，仍然不得不承认，青春说到底，终究还是一本写得过于仓促的书。

写到这里，才更加真切地意识到，本科四年的时光确实已经走到了尾声。过去总觉得日子还很长，长到可以从容地完成每一次告别，也可以慢慢地学会成长，还可以把那些一直没有说出口的话逐一安放妥当。可是，当真正回过头去看这一路时才发现，岁月从来不会为任何人停下脚步；那些普通的清晨以及黄昏，那些并肩走过的道路，以及那些曾以为还会不断重逢的相遇，早已在不知不觉当中，沉淀成生命里温柔而珍贵的注脚。即使心中仍然存有许多不舍，也终究需要把行囊整理好，在告别里继续向前出发，去奔赴下一段旅程。

回望一路求学的经历，写到恩师这一页时，最适宜落下的句子，仍是“人师难遇，如沐春风”。从论文选题一次次进行斟酌，到研究思路逐渐变得清楚；从整体框架一点点搭建起来，到文字细节反复进行推敲，每一个环节当中，都凝结着郑积仕老师细致而耐心的指导与点拨。郑老师严谨求实的治学态度，以及宽厚温和的育人风范，使迷茫之时可以辨明方向，也使踌躇之际能够增添继续前行的勇气。同时，也诚挚感谢系主任赵钊林老师在专业学习、学业规划以及毕业相关工作中给予了许多耐心指导，使我能够更加清楚地认识专业方向与学习目标；吕福春老师在班级事务、日常学习以及成长过程中始终给予关心与提醒，使我在大学生活中感受到踏实而温暖的支持；也感谢所有曾给予教导、鼓励以及支持的老师们。正是诸位以知识为灯，以言传身教为路，才使这段求学之途能够在不断成长当中稳步向前。愿诸位老师杏坛常暖，桃李芬芳，教泽绵长。

念及至亲，写到家人这一页，最适宜落笔的，仍是“椿萱并茂，庭闱常暖”。年岁渐长，才越发明白，有些爱正因为太过深厚，反倒不容易轻易说出口。二十二载春秋，一屋五人，三餐四季，早已把最深的惦念，悄悄放进每一次叮咛、每一顿热饭以及每一次等候之中。虽然并不习惯在家人面前直白表达爱意，却一直明白，不论将来走到多远，家都会是回望时最柔软、也最安稳的归处。感谢父母一路托举，也感谢爷爷奶奶始终疼爱，是你们给予奔向更大世界的底气，也让人始终记得，来处一直有光。长大也许并不总是轻松，心底仍会怀念那个被全家人宠着、无忧无虑的小孩。惟愿家人平安顺遂，岁岁康健，仍如儿时所愿，长久相伴。

念及挚友，写到朋友这一页，最适宜落下的句子，仍是“芝兰同芳，管鲍相契”。人生这段旅途当中，难免会遇见失落与疲惫，所幸总有人愿意在情绪低落时安静倾听，在心中犹疑时给予真诚回应，在收获欢喜时也由衷地一同雀跃。感谢一路同行的朋友们，感谢你们一次次接住情绪、包容不够成熟的笨拙，也陪伴着走过许多平凡却明亮的时刻。那些说走就走的旅行、深夜里的倾诉、突如其来的大笑，以及彼此认真鼓励的瞬间，都会在岁月

当中沉淀为青春里不可替代的珍贵片段。愿大家在各自的人生道路上继续发光，前程似锦，所愿皆得；也愿多年以后再次回望，仍能真切地说一句：幸好当初遇见了你们。

念及自身，写到自己这一页，最适宜落笔的，仍是“明媚自由，步履生花”。感谢那个一路跌跌撞撞却始终不曾真正放弃的自己，也感谢自己在许多想要停下以及退缩的时刻，仍然选择咬牙向前；在那些充满不确定与迷惘的日子里，依然愿意认真相信远方，也相信未来。愿往后的自己，仍有奔赴热爱的勇气，也保有顺其自然的从容；仍能真诚地感受世界，坦荡地接纳自己，在爱与被爱之间，慢慢成为更加完整而坚定的人。

这篇文章最终在五月慢慢收笔，窗外草木正一寸一寸地繁盛起来，而属于人生的春天，也不过才刚刚开始启程。于是，便带着一颗安静而笃定的心，去等枝桠继续舒展、繁花如约盛开的那一天。谨以此文，献给热烈、真挚并且独一无二的二十二岁。